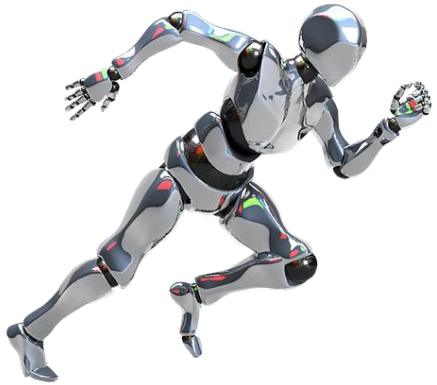
Optimization
最佳化

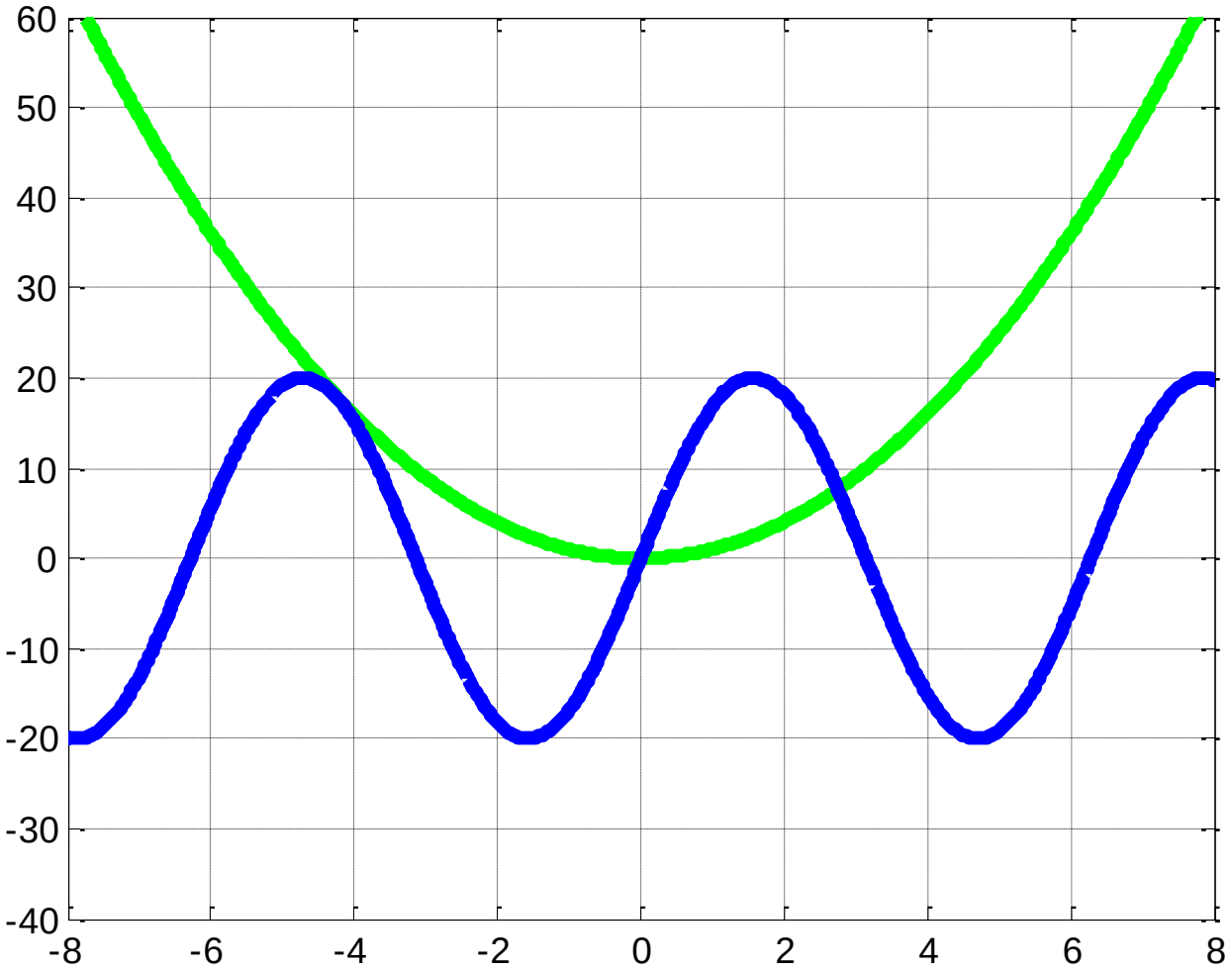
高怡宣
陽明大學 生物醫學影像暨放射科學系



1. Gradient Descent
2. Constraint
3. $f(x_i, w_i, b) = \sum_{i=1}^{n} x_i w_i + b$

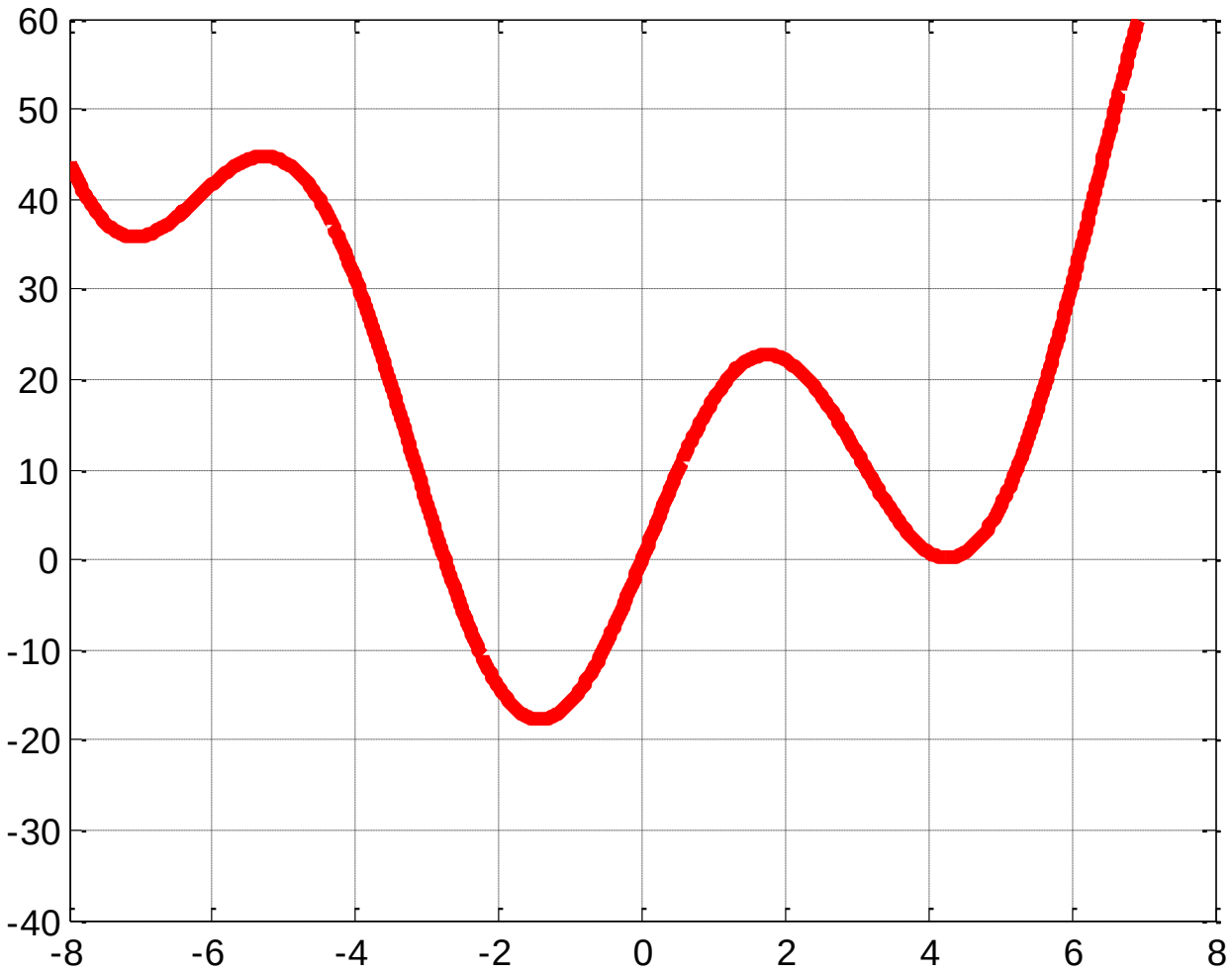
One Dimensional gradient decent:

$$minimize \quad f(x) = x^2 + 20\, sin(x)$$

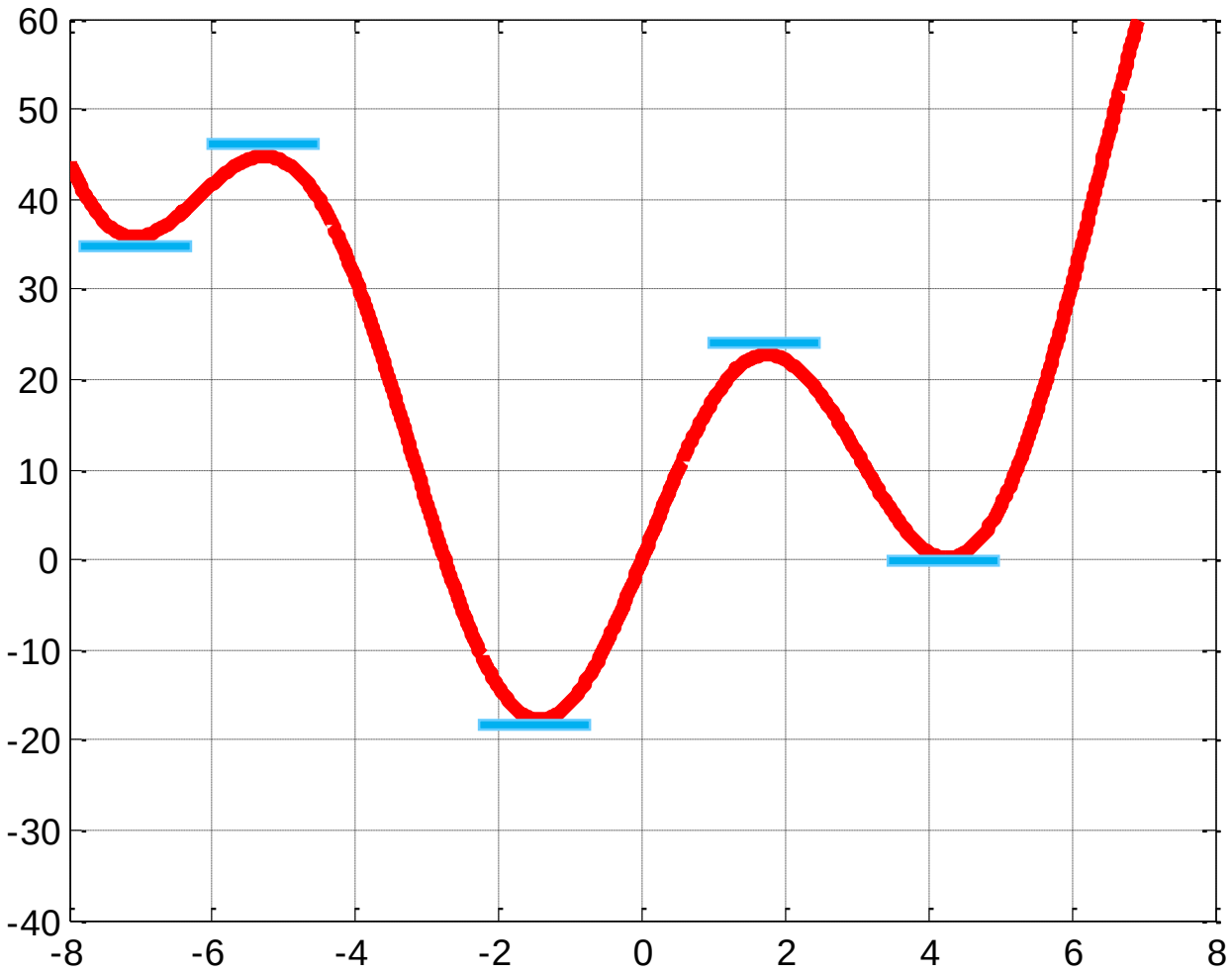


$$f(x) = x^2$$

$$f(x) = 20sin(x)$$



$$f(x) = x^2 + 20\,sin(x)$$

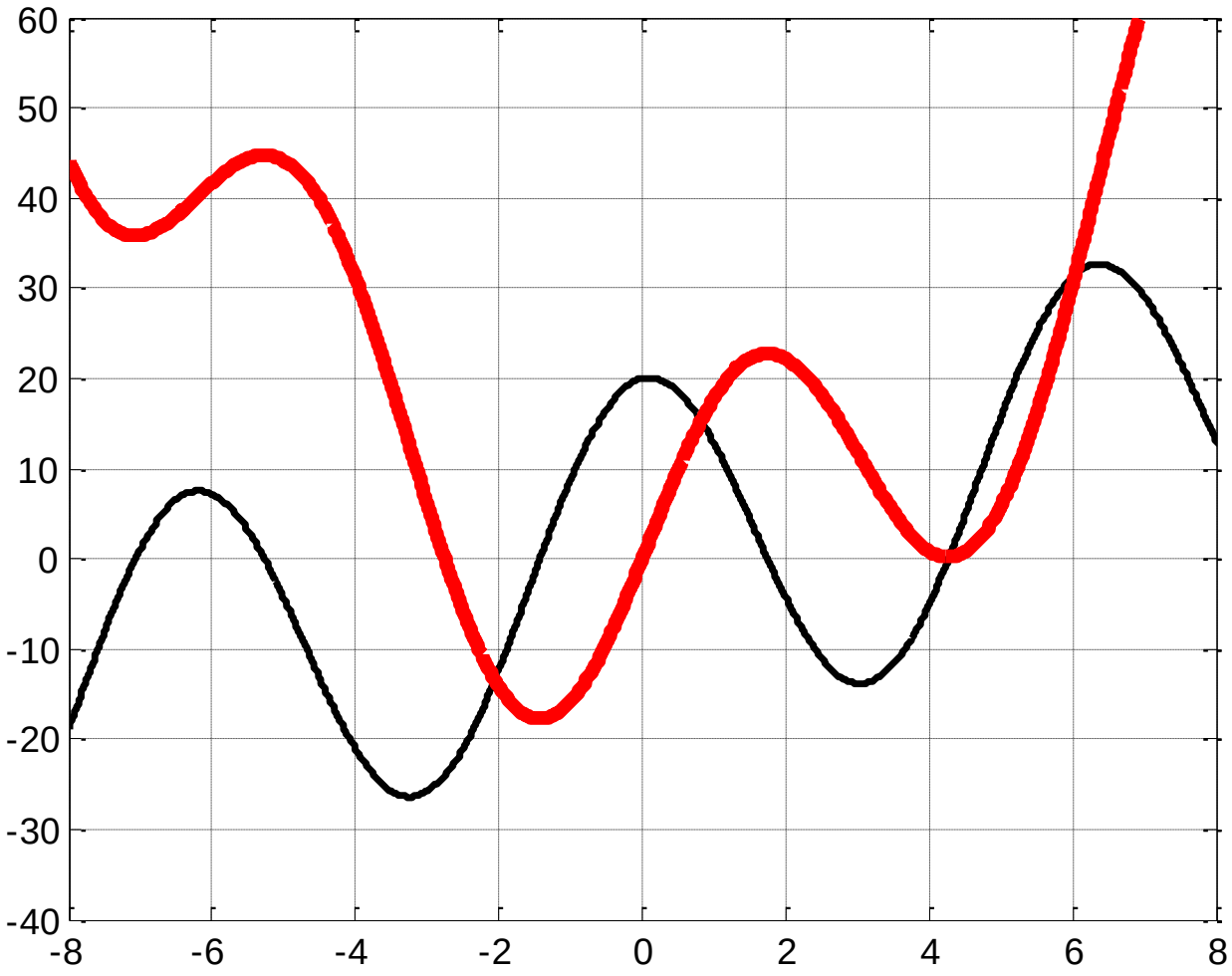


$$f(x) = x^2 + 20\,sin(x)$$

For a function: $f(x) = x^2 + 20\,sin(x)$

The minimums occur at $\frac{df}{dx} = f'(x) = 0$

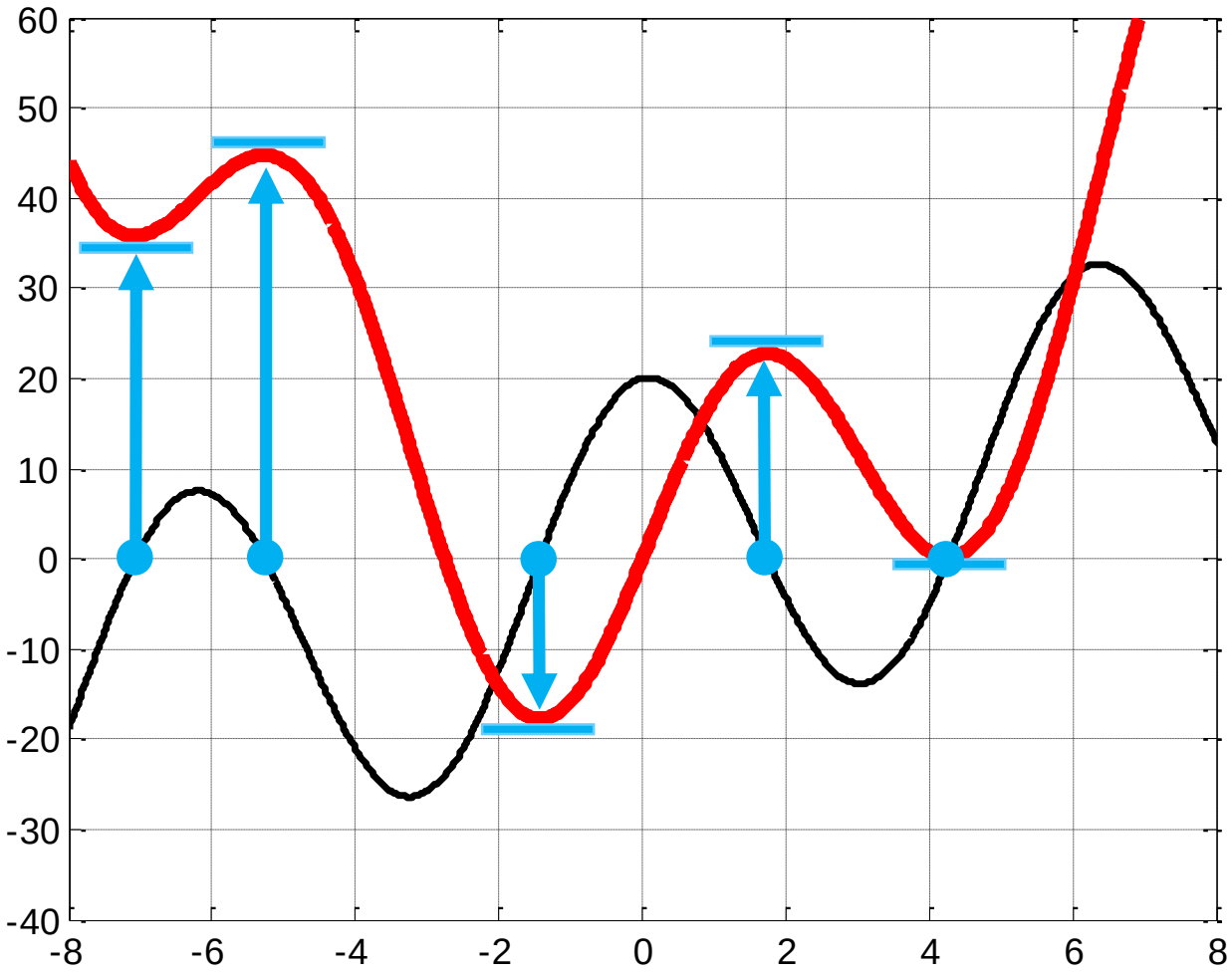
That is: $\frac{df}{dx} = f'(x) = 2x + 20\,cos(x)$



$$f(x) = x^2 + 20\,sin(x)$$

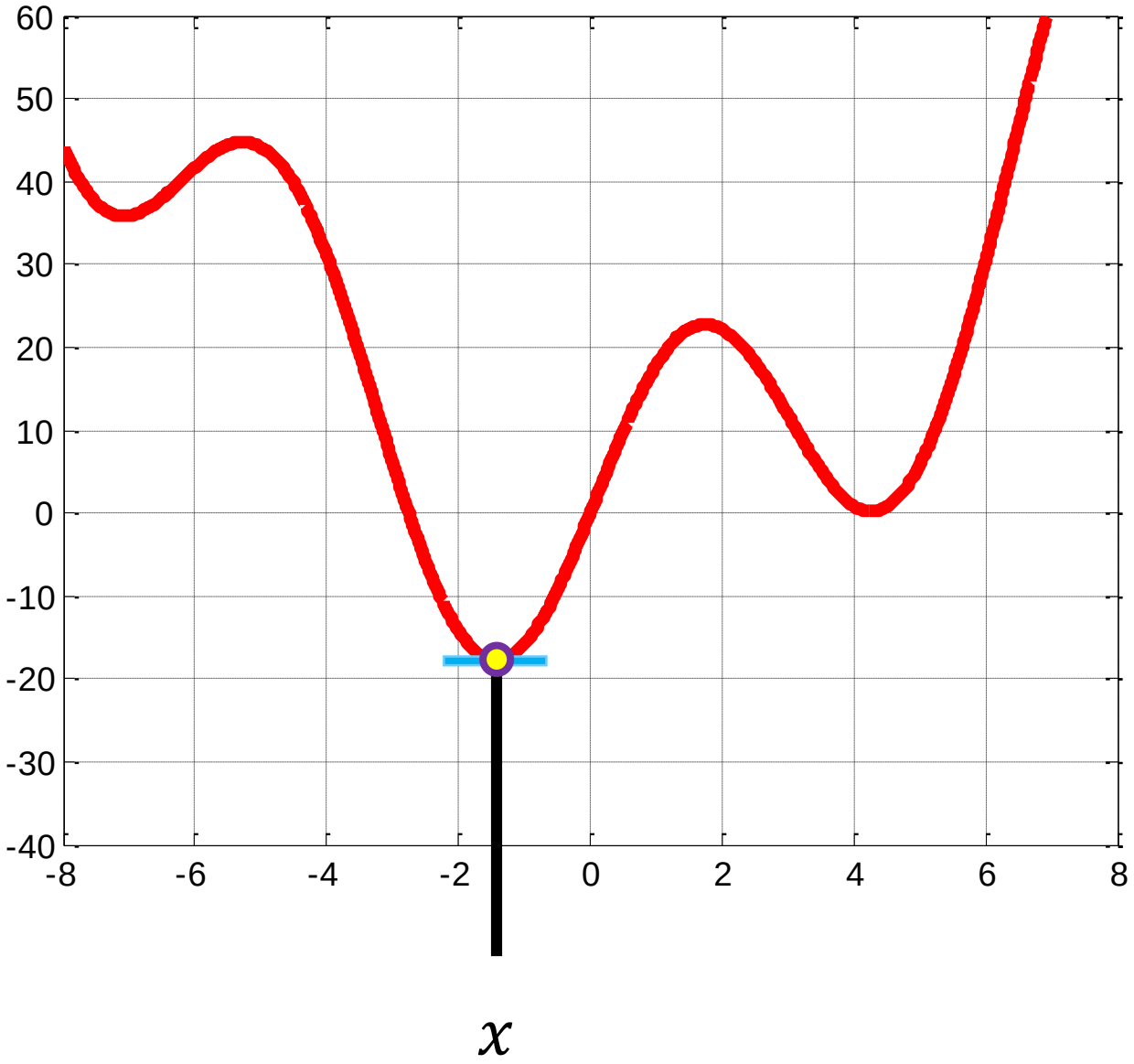
$$f'(x) = 2x + 20cos(x)$$

f'(x)和粒子加速器 Longitudinal motion 的公式類似, 請看以下影片 (from 56:28 to 1:01:16)
https://www.youtube.com/watch?v=SErkz22xZ_U&index=15&list=PLYgGtiVoYLPdIU1WLQSd1KDi9jF-K_0Ds



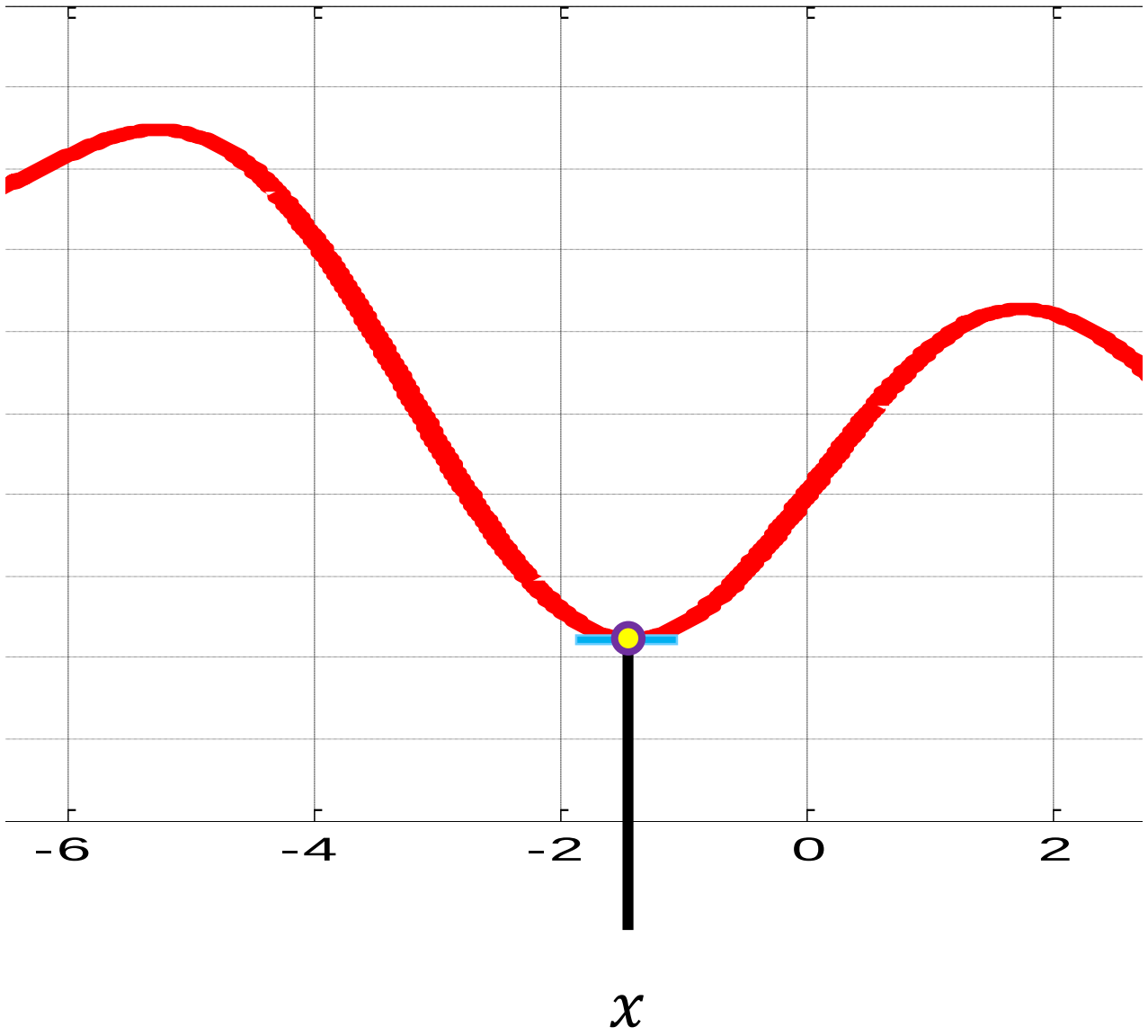
$$f(x) = x^2 + 20\,sin(x)$$

$$f'(x) = 2x + 20cos(x)$$



$$f(x) = x^2 + 20\,sin(x)$$

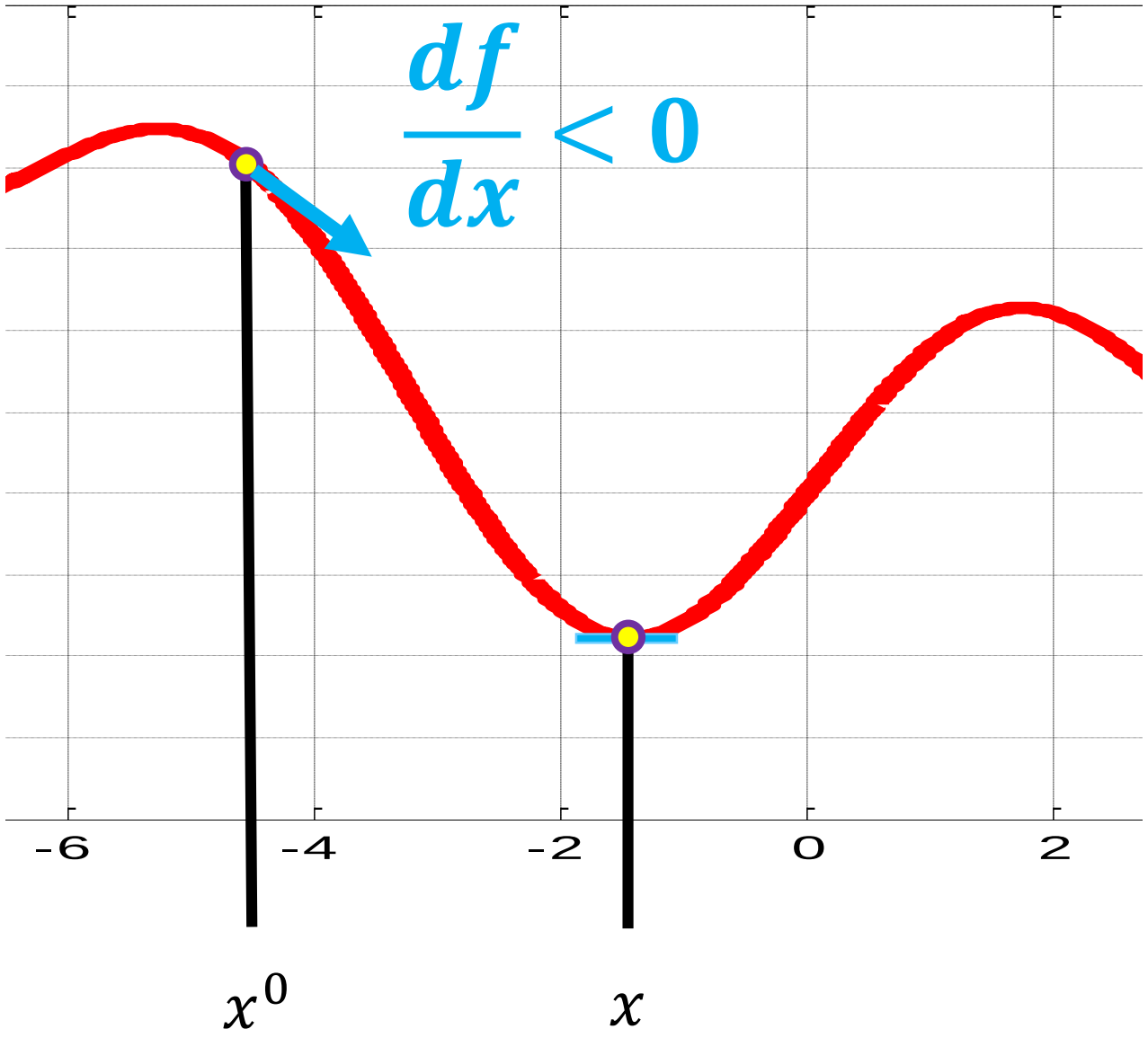
$$Mission: find\ x$$



$$f(x) = x^2 + 20\,sin(x)$$

$$\frac{df}{dx} = f'(x) =$$

$$2x + 20cos(x)$$



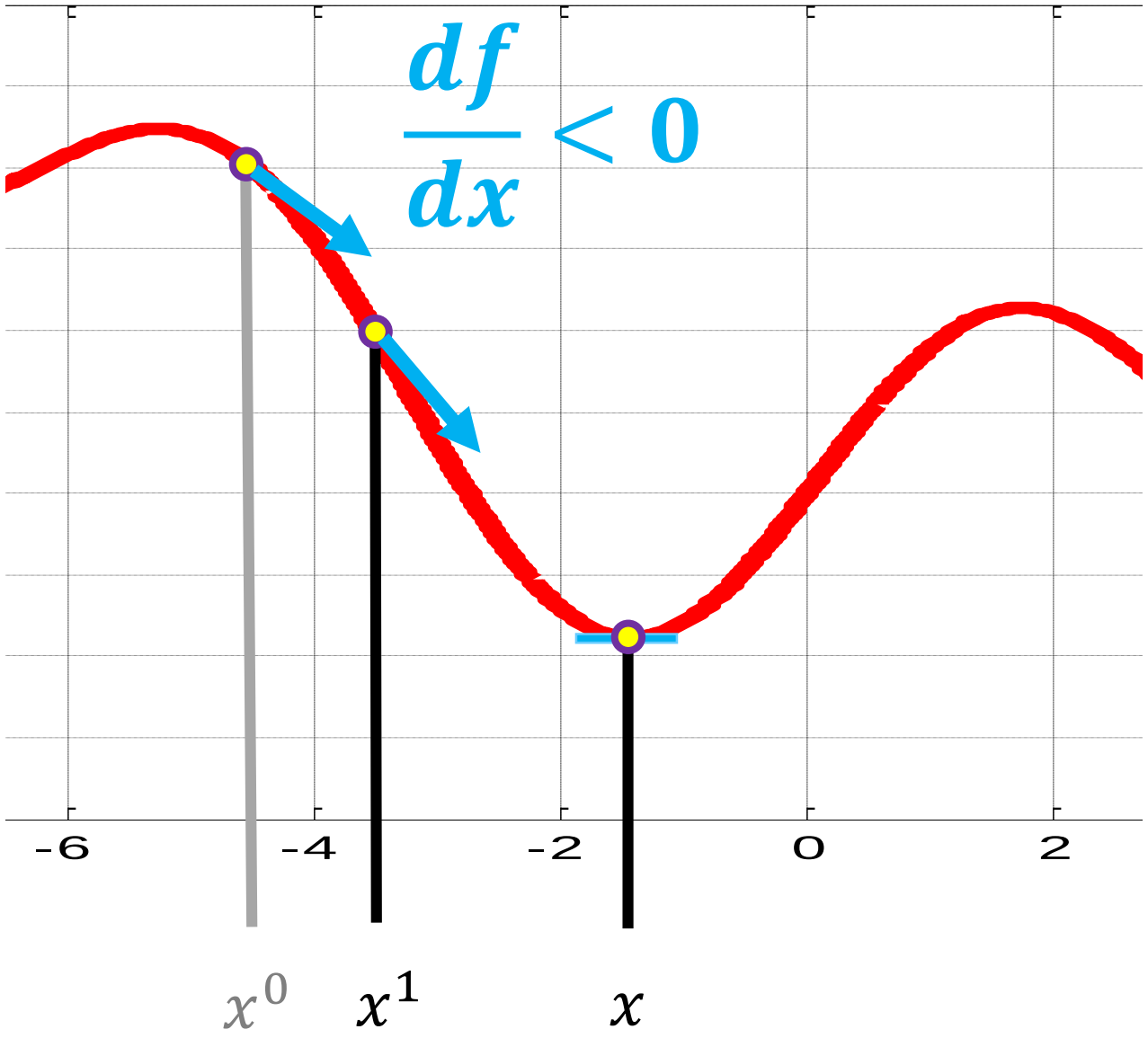
$$f(x) = x^2 + 20\,sin(x)$$

x^0 : *initial guess*

$$x^1 = x^0 - \eta \left.\frac{df}{dx}\right|_{x = x^0}$$

η: *learning rate*

$\eta > 0$

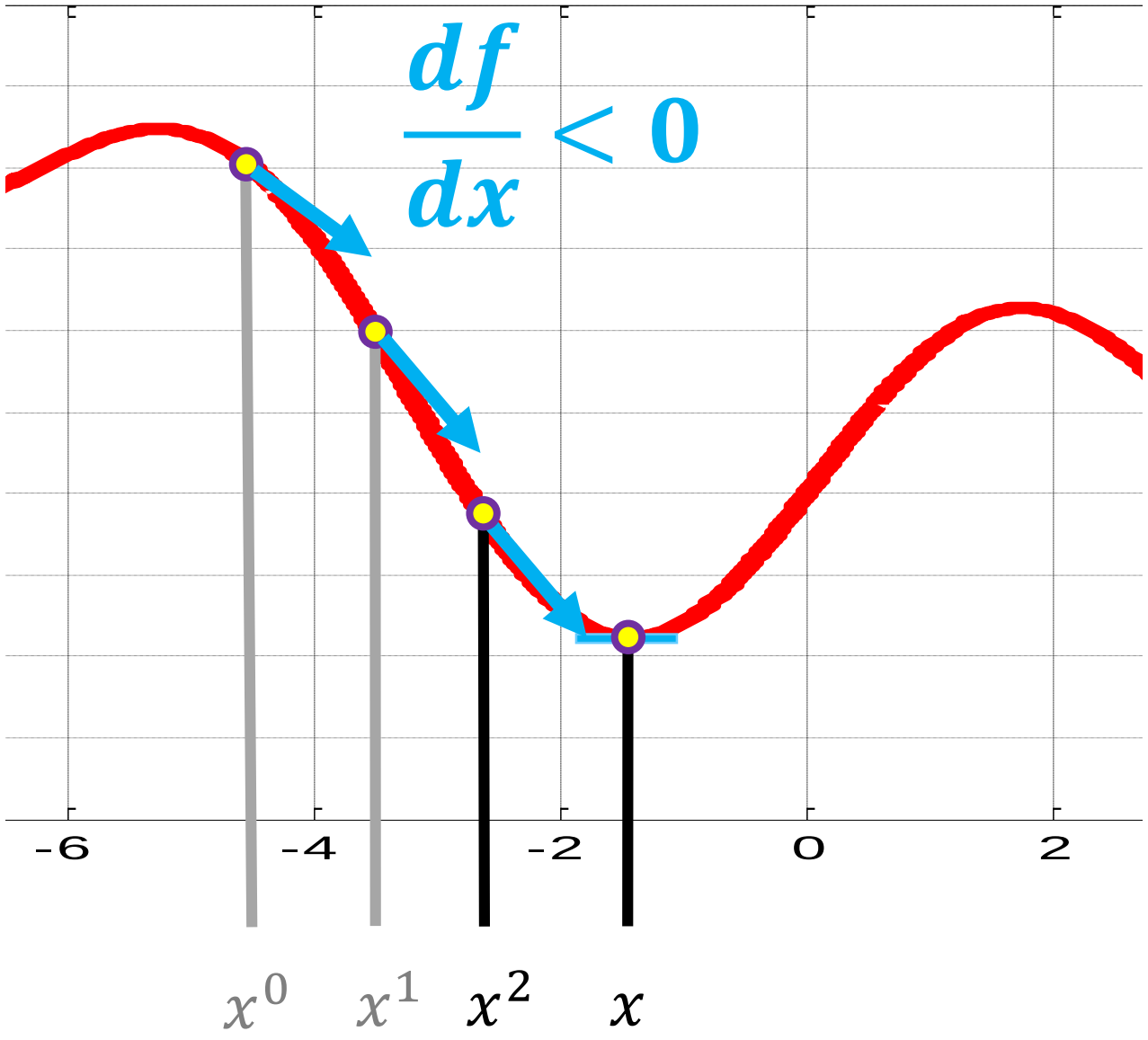


$$f(x) = x^2 + 20\,sin(x)$$

x^0 : *initial guess*

$$x^1 = x^0 - \eta \left.\frac{df}{dx}\right|_{x\,=\,x^0}$$

$$x^2 = x^1 - \eta \left.\frac{df}{dx}\right|_{x\,=\,x^1}$$



$$f(x) = x^2 + 20\,sin(x)$$

x^0 : *initial guess*

.

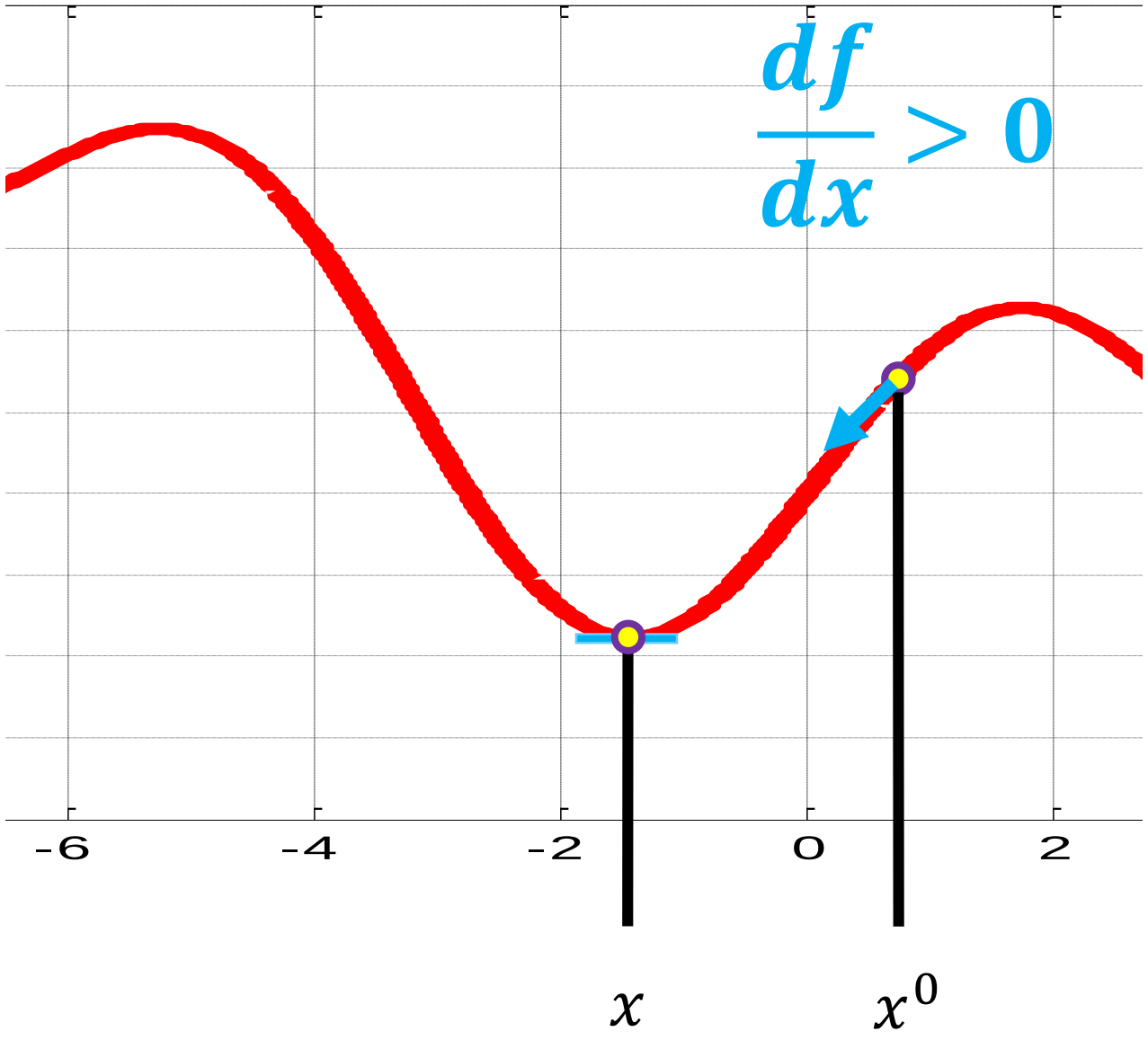
.

.

$$x^n = x^{n-1} - \eta \left.\frac{df}{dx}\right|_{x^{n-1}}$$

$$until\ \left|x^n - x^{n-1}\right| < \varepsilon$$

From the other side:



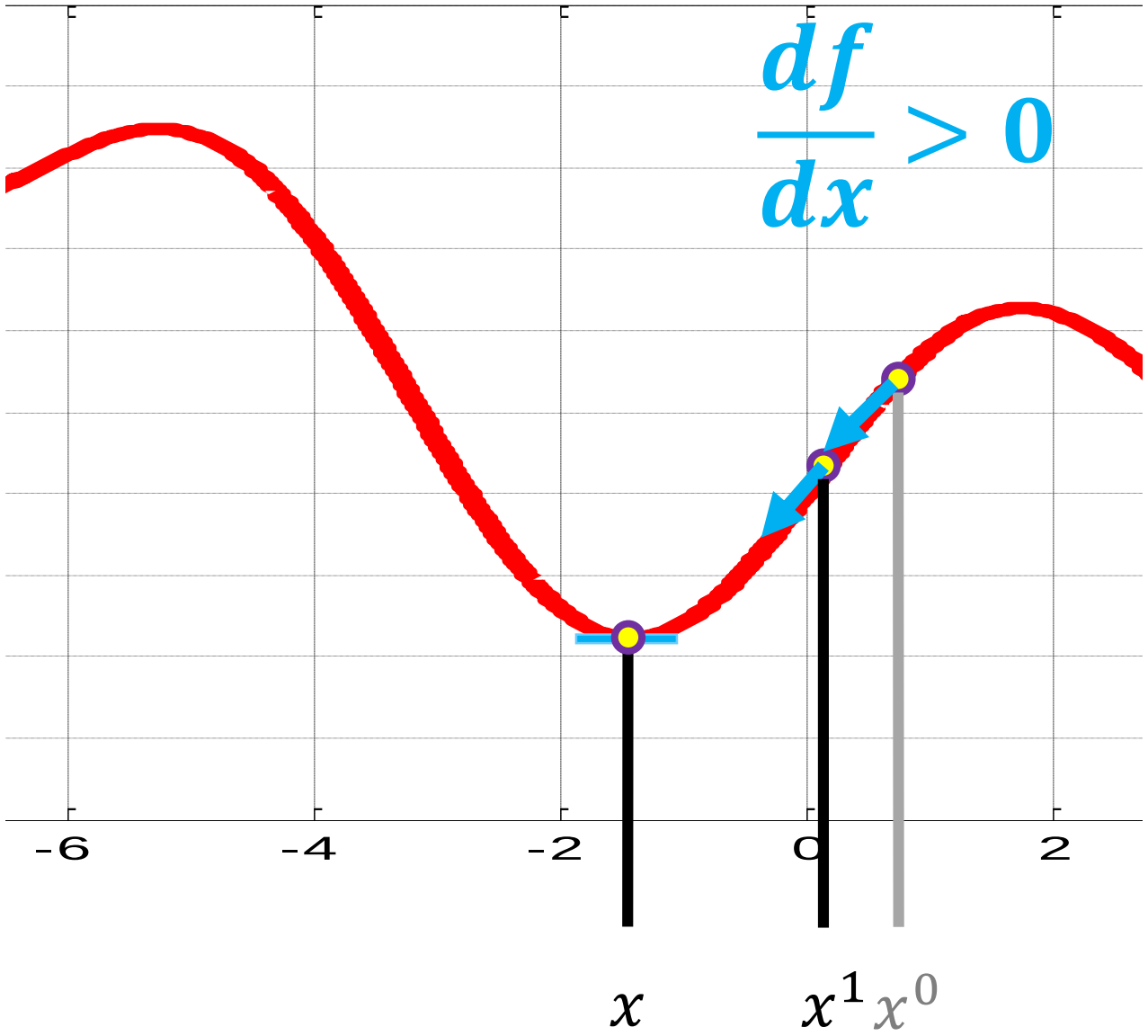
$$f(x) = x^2 + 20\,sin(x)$$

x^0 : *initial guess*

$$x^1 = x^0 - \eta \left.\frac{df}{dx}\right|_{x = x^0}$$

η: *learning rate*

$\eta > 0$

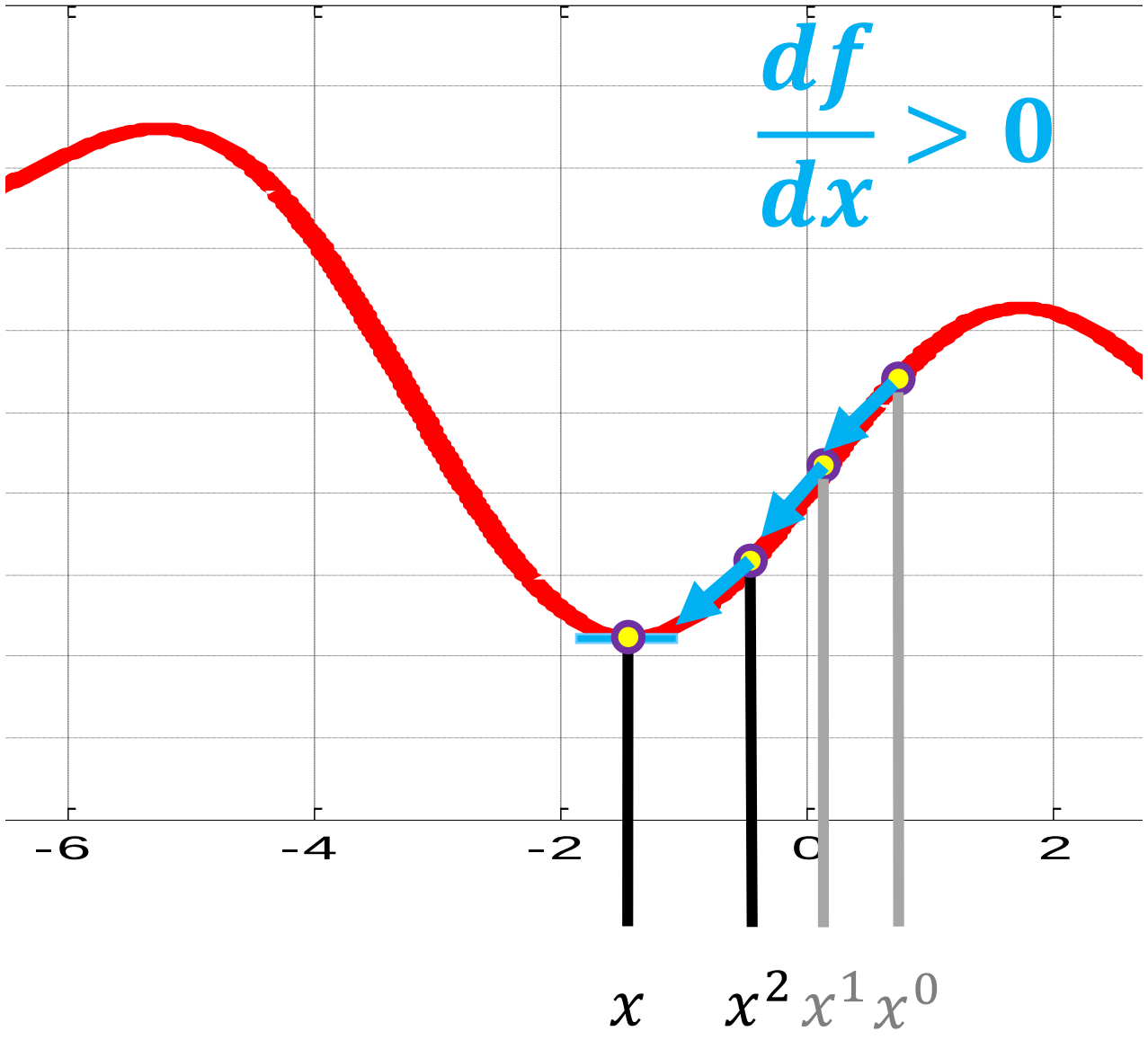


$$f(x) = x^2 + 20\,sin(x)$$

x^0 : *initial guess*

$$x^1 = x^0 - \eta \frac{df}{dx}\bigg|_{x\,=\,x^0}$$

$$x^2 = x^1 - \eta \frac{df}{dx}\bigg|_{x\,=\,x^1}$$



$$f(x) = x^2 + 20\,sin(x)$$

x^0 : *initial guess*

.

.

.

$$x^n = x^{n-1} - \eta \left.\frac{df}{dx}\right|_{x^{n-1}}$$

$$until\ \left|x^n - x^{n-1}\right| < \varepsilon$$

An example:

$$f(x) = x^2 + 20\,sin(x)$$

jupyter Python_Gradient_Descent_One_Dimen

File Edit View Insert Cell Kernel Widgets Help

Run Markdown

Gradient descent: f(x) = x^^2 + 20*sin(x)

(1) import libraries

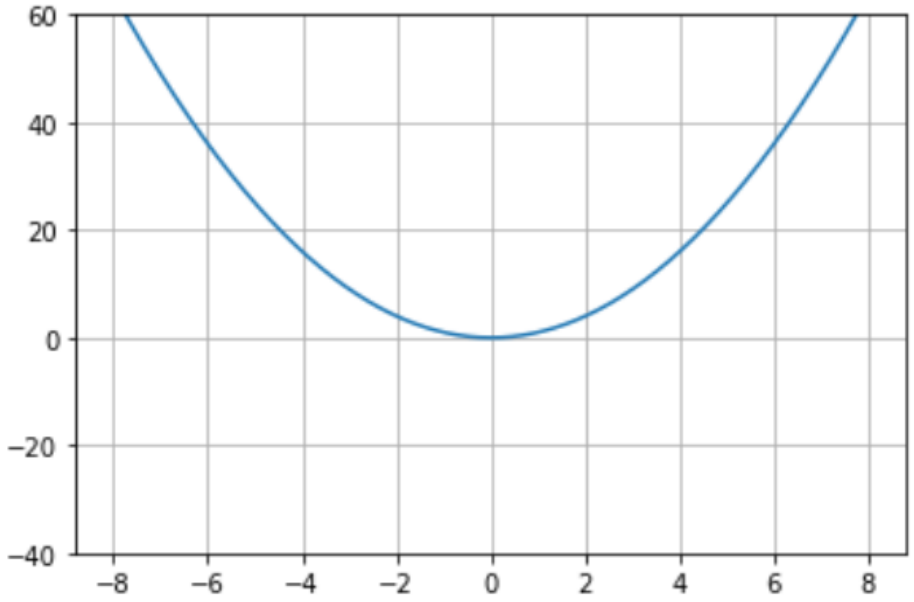
In [1]:
```
import math
import numpy as np
import matplotlib.pyplot as plt
```

(2) set up x and y as array

In [2]:
```
x = np.zeros(1600)
y = np.zeros(1600)
print(x.dtype, x.size, y.dtype, y.size)
```

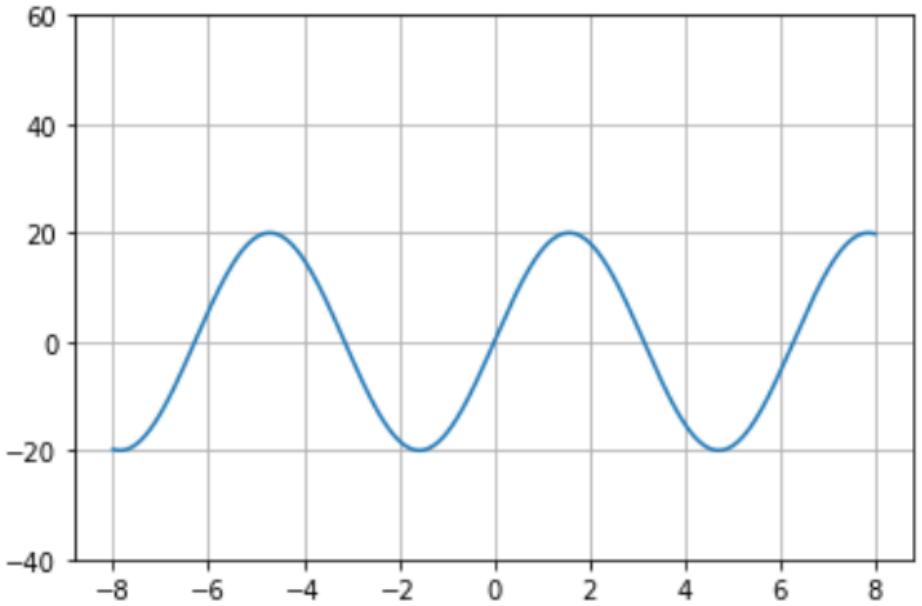
(3) f(x) = x^^2

In [3]:
```
for i in range(0,1600):
    x[i]=i/100 - 8
    y[i]=math.pow(x[i],2)
plt.plot(x,y)
plt.ylim([-40,60])
plt.grid()
```



(4) f(x) = 20*sin(x)

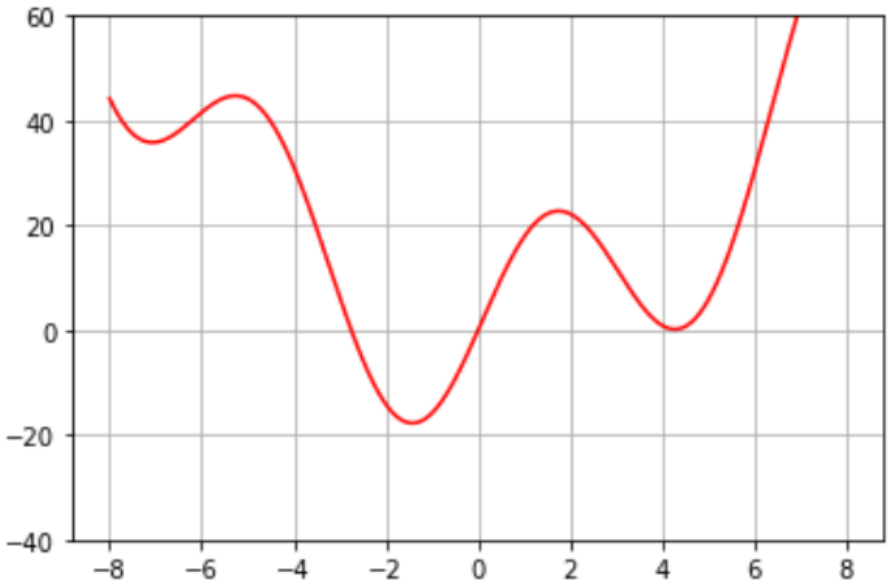
In [4]:
```
for i in range(0,1600):
    x[i]=i/100 - 8
    y[i]=20*math.sin(x[i])
plt.plot(x,y)
plt.ylim([-40,60])
plt.grid()
```



(5) f(x) = x^^2 + 20*sin(x)

In [5]:
```
for i in range(0,1600):
     x[i]= i/100 - 8
     y[i]= math.pow(x[i],2)+20*math.sin(x[i])

plt.plot(x,y,color='red')
plt.ylim([-40,60])
plt.grid()
```

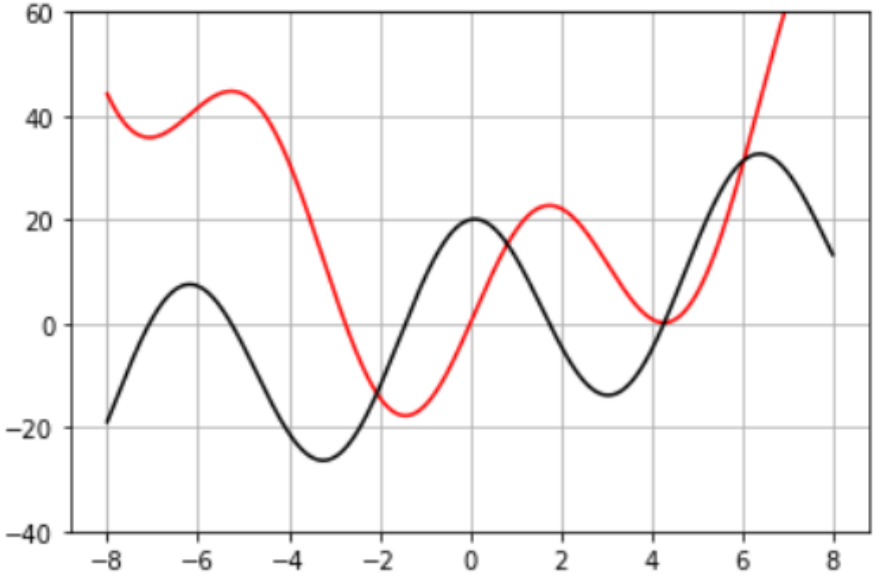


(6) df/dx = f'(x) = 2*x* + *20*cos(x)

In [6]:
```
df = np.zeros(1600)

for i in range(0,1600):
     x[i]= i/100 - 8
     y[i]= math.pow(x[i],2)+20*math.sin(x[i])
     df[i] = 2*(x[i])+20*math.cos(x[i])

plt.plot(x,y,color='red')
plt.plot(x,df,color='black')
plt.ylim([-40,60])
plt.grid()
plt.show()
```



(7) Set up the initial value, start at cur_x :

In [7]:
```
cur_x = -4 # The algorithm starts at x = -4
rate = 0.01 # Learning rate
precision = 0.001 #This tells us when to stop
previous_step_size = 1 #
max_iters = 50 # maximum number of iterations
iters = 0 #iteration counter
df = lambda x: 2*x+20*math.cos(x) #Gradient o
```

https://towardsdatascience.com/implement-gradient-descent-in-python-9b93ed7108d1

(8) Do the iteration

In [8]:
```
while previous_step_size > precision and iters < max_iters:
    prev_x = cur_x #Store current x value in prev_x
    cur_x = cur_x - rate * df(prev_x) #Grad descent
    previous_step_size = abs(cur_x - prev_x) #Change in x
    iters = iters+1 #iteration count
    print("Iteration",iters,"\nX value is",cur_x) #Print it

print("The local minimum occurs at", cur_x)
```

https://towardsdatascience.com/implement-gradient-descent-in-python-9b93ed7108d1

```
Iteration 15
X value is -1.555018893605061
Iteration 16
X value is -1.5270738714580432
Iteration 17
X value is -1.5052740992902973
Iteration 18
X value is -1.4882636882327407
Iteration 19
X value is -1.4749862091508736
Iteration 20
X value is -1.4646192053990756
Iteration 21
X value is -1.4565223681752868
Iteration 22
X value is -1.4501970033103802
Iteration 23
X value is -1.4452545031011257
Iteration 24
X value is -1.4413918752917774
Iteration 25
X value is -1.4383727570799512
Iteration 26
X value is -1.436012677723964
Iteration 27
X value is -1.434167609256578
Iteration 28
X value is -1.4327250631025399
Iteration 29
X value is -1.4315971599869464
Iteration 30
X value is -1.430715231216223
The local minimum occurs at -1.430715231216223
```


```
Iteration 30
X value is -1.430715231216223
The local minimum occurs at -1.430715231216223
```

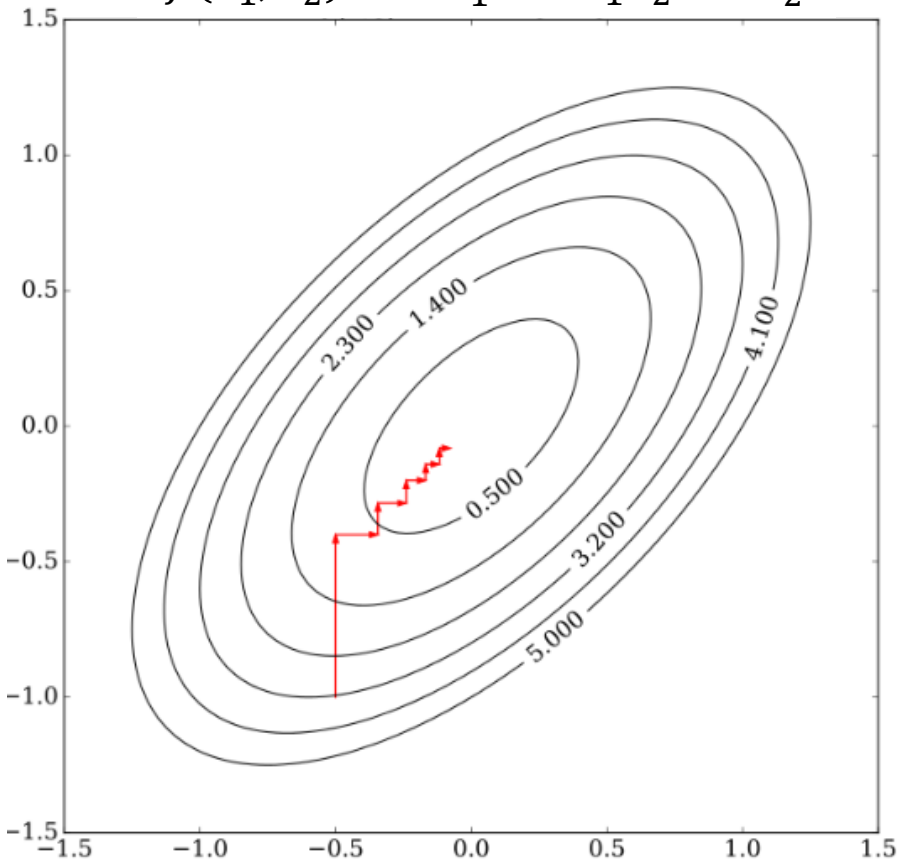
Algorithm 1 Gradient Descent

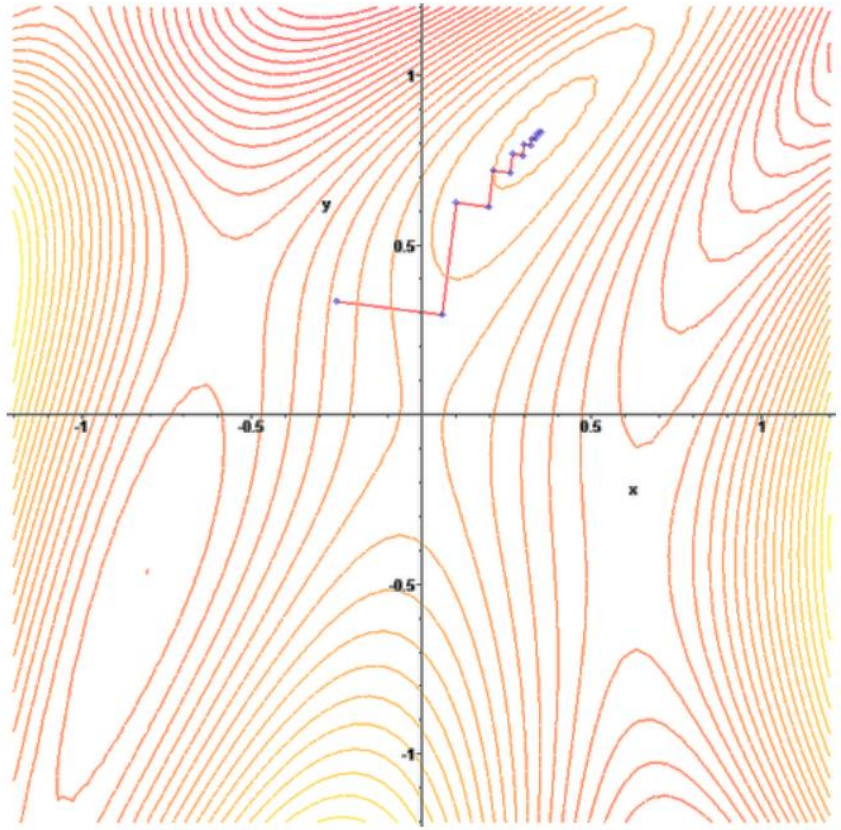
Guess $\mathbf{x}^{(0)}$, set $k \leftarrow 0$
while $||\nabla f(\mathbf{x}^{(k)})|| \geq \epsilon$ **do**
    $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - t_k \nabla f(\mathbf{x}^{(k)})$
    $k \leftarrow k + 1$
end while
return $\mathbf{x}^{(k)}$

Prof. Yaron Singer, https://people.seas.harvard.edu/~yaron/AM221-S16/sections/sec2.pdf

Two Dimensional Gradient Descent

$$f(x_1, x_2) = 5x_1^2 - 6x_1x_2 + 5x_2^2$$





Nicoguaro, https://commons.wikimedia.org/w/index.php?curid=46260701

https://en.wikipedia.org/wiki/Gradient_descent

Example. Consider the problem of minimizing $f(x,y) = 4x^2 - 4xy + 2y^2$ using the gradient descent method. Notice that the optimal solution is $(x,y) = (0,0)$. To apply the gradient descent algorithm let's first compute the gradient:

$$\nabla f(x,y) = \begin{pmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{pmatrix} = \begin{pmatrix} 8x - 4y \\ -4x + 4y \end{pmatrix}$$

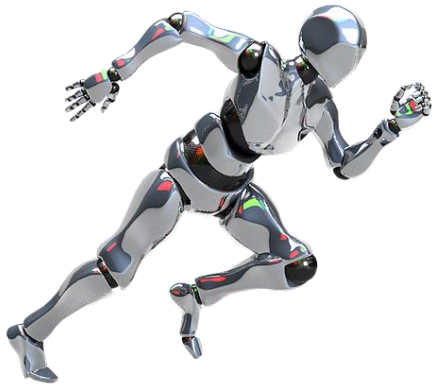
We will start from the point $(x^{(0)}, y^{(0)}) = (2,3)$. To find the next point $(x^{(1)}, y^{(1)})$ we compute:

$$(x^{(1)}, y^{(1)}) = (x^{(0)}, y^{(0)}) - t_0^\star \nabla f(x^{(0)}, y^{(0)}).$$

$$\boxed{x^1 = x^0 - \eta \left.\frac{\partial f}{\partial x}\right|_{x^0, y^0}}$$

$$\boxed{y^1 = y^0 - \eta \left.\frac{\partial f}{\partial y}\right|_{x^0, y^0}}$$

Prof. Yaron Singer, https://people.seas.harvard.edu/~yaron/AM221-S16/sections/sec2.pdf



1. Gradient Descent
2. Constraint
3. $f(x_i, w_i, b) = \sum_{i=1}^{n} x_i w_i + b$

$$Minimize$$

$$f(w_1, w_2) = 5w_1^2 - 6w_1w_2 + 5w_2^2$$

$$subject\ \ to\ \ w_1^2 + 4w_2^2 = 1$$ *(constraint)*

$$Minimize$$

$$f(w_1, w_2) = 5w_1^2 - 6w_1w_2 + 5w_2^2$$

$$subject\ \ to\ \ w_1^2 + 4w_2^2 = 1$$

$$that\ is,\ \ w_1^2 + 4w_2^2 - 1 = 0$$

$$Minimize$$

$$f(w_1, w_2) = 5w_1^2 - 6w_1w_2 + 5w_2^2$$

$$subject\ \ to\ \ w_1^2 + 4w_2^2 = 1$$

$$that\ is,\ \ w_1^2 + 4w_2^2 - 1 = 0$$

$$that\ is,\ \ \lambda\,(w_1^2 + 4w_2^2 - 1) = 0$$

Define a Lagrange function:

$$L(w_1, w_2, \lambda) = f(w_1, w_2) + \lambda(\cdots)$$

λ is the Lagrange multiplier

https://en.wikipedia.org/wiki/Lagrange_multiplier

Define a Lagrange function:

$$L(w_1, w_2, \lambda) = f(w_1, w_2) + \lambda(\cdots)$$

$$L(w_1, w_2, \lambda) = 5w_1^2 - 6w_1w_2 + 5w_2^2$$

$$+ \lambda\,(w_1^2 + 4w_2^2 - 1)$$

Choose η *and* λ

Guess w_1^0 *and* w_2^0

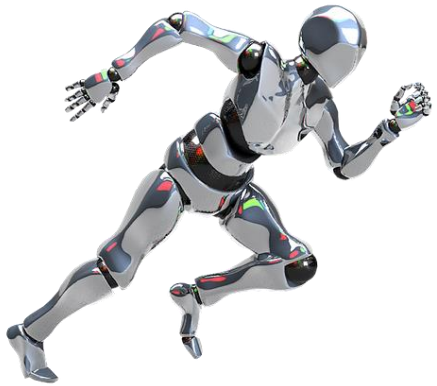
Do iterations with

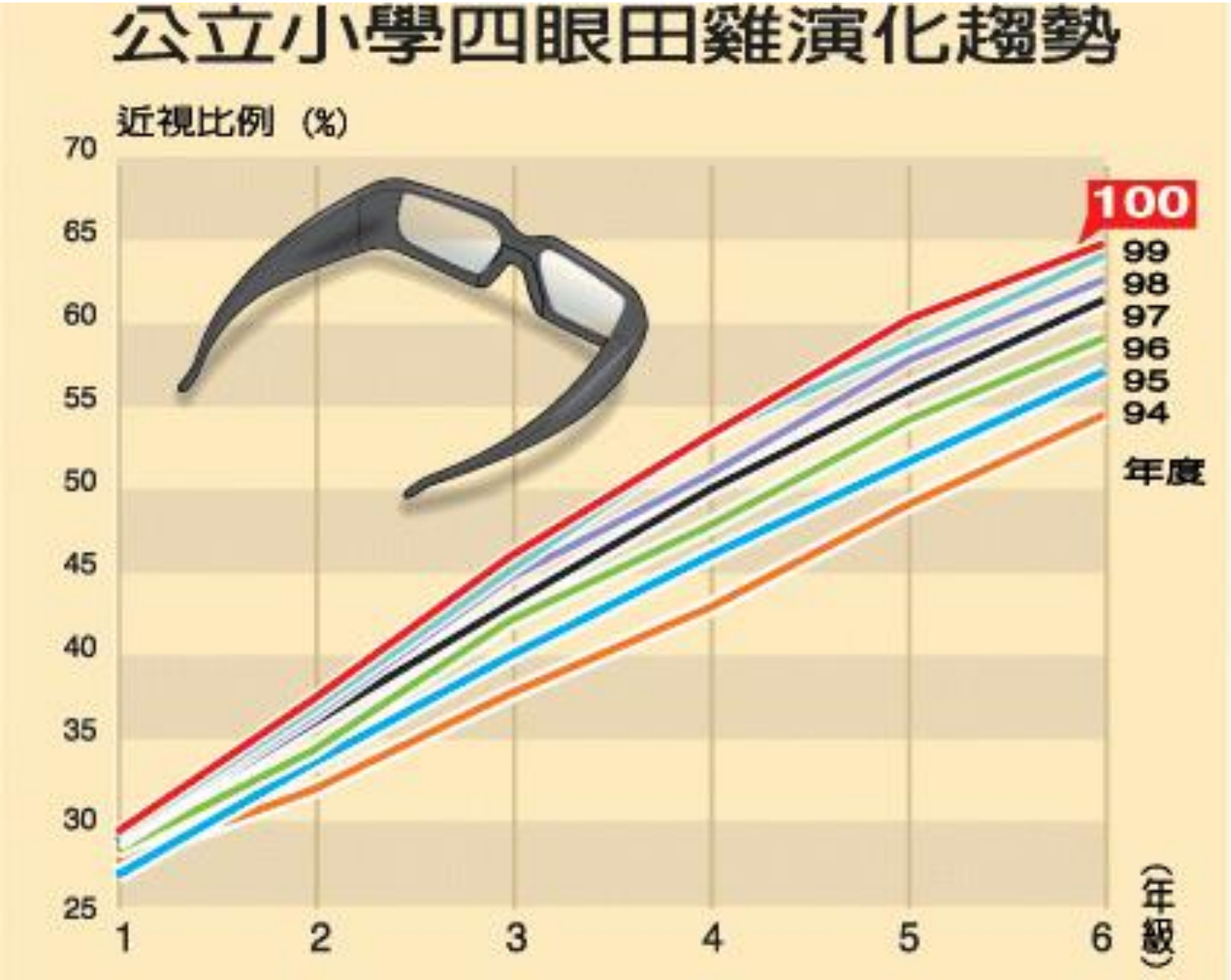
$$\boxed{w_1^1 = w_1^0 - \eta \frac{\partial L(w_1, w_2, \lambda)}{\partial w_1}}$$

$$\boxed{w_2^1 = w_2^0 - \eta \frac{\partial L(w_1, w_2, \lambda)}{\partial w_2}}$$

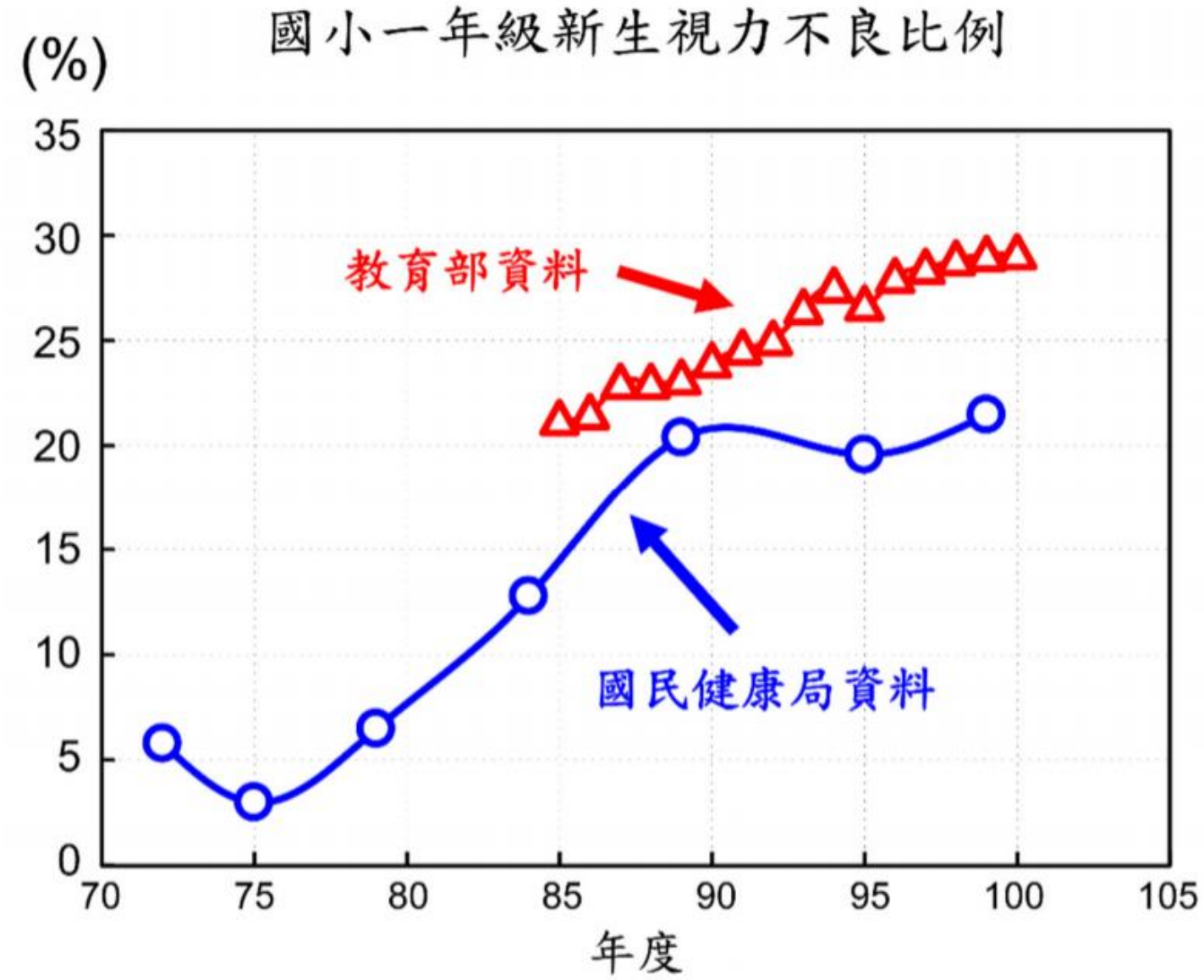
Stop when change is small or limit is reached

1. Gradient Descent
2. Constraint
3. $f(x_i, w_i, b) = \sum_{i=1}^{n} x_i w_i + b$





https://sites.google.com/view/yhkao/報紙投書/ipad-取代教科書-搶救四眼小低頭族



https://sites.google.com/view/yhkao/報紙投書/兒童節最好的禮物-視力急惡化-搶救小四眼田雞

中華民國統計資訊網
National Statistics

國小學生視力不良率

- 變數與數值

編輯與計算 | 另存新檔 | 繪圖與地圖
捷徑 | XLS ODS CSV PX M+ | 繪圖教學

學生視力不良率-年 依 期間, 指標, 年級別 與 性別

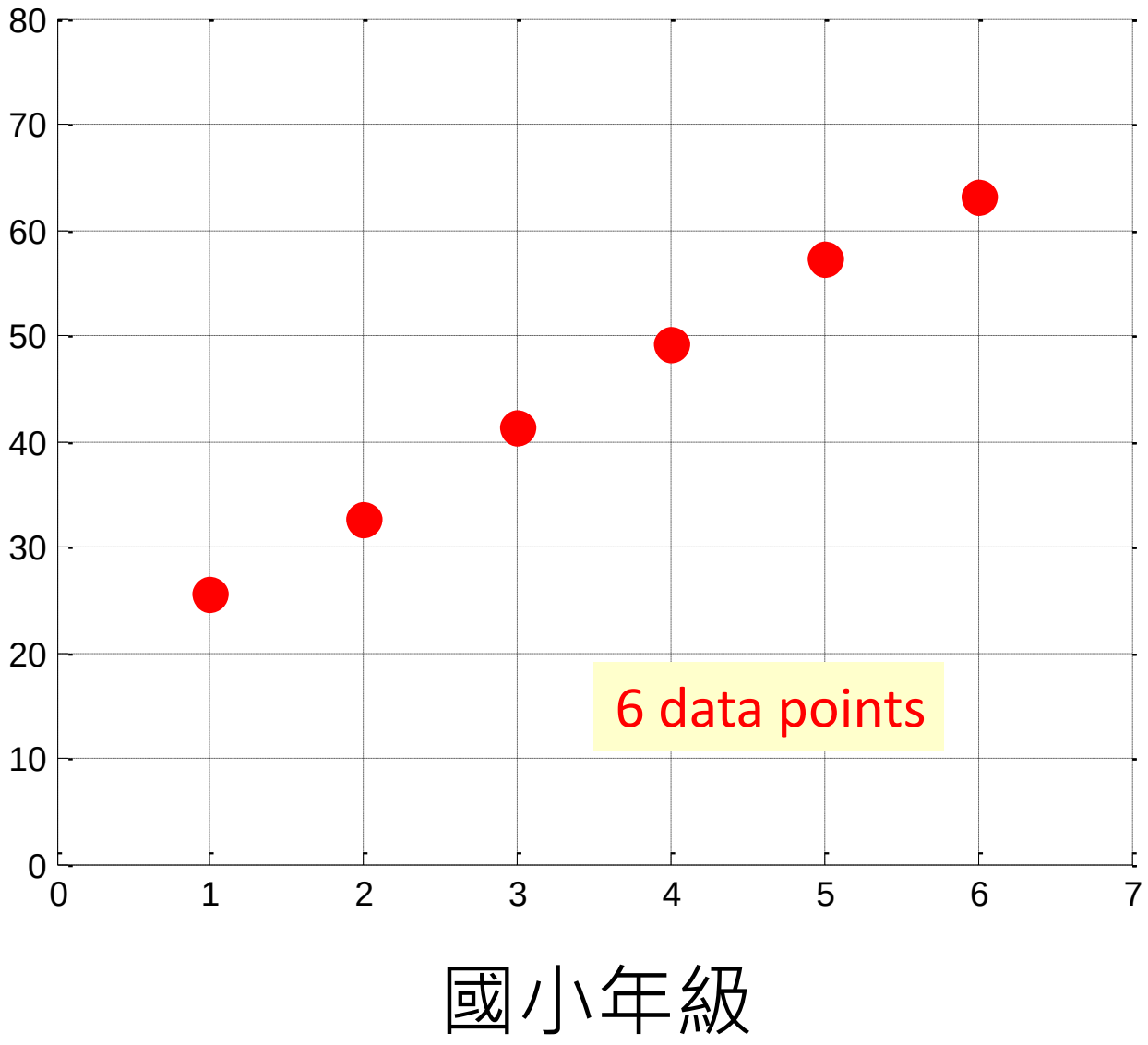
	國小一年級			國小二年級			國小三年級			國小四年級			國小五年級			國小六年級		
	計	男	女	計	男	女	計	男	女	計	男	女	計	男	女	計	男	女
2017	25.64	25.11	26.22	32.71	32.39	33.07	41.33	41.23	41.44	49.13	48.31	50.04	57.21	55.90	58.64	63.21	61.20	65.43

註解：
more
國情統計通報
1.兩眼視力均達0.9以上者為視力正常；一眼在0.9以下者為視力不良。
2.高中(職)自2014學年度起加入實用技能學程學生。

最近更新日：2018-04-30 18:00

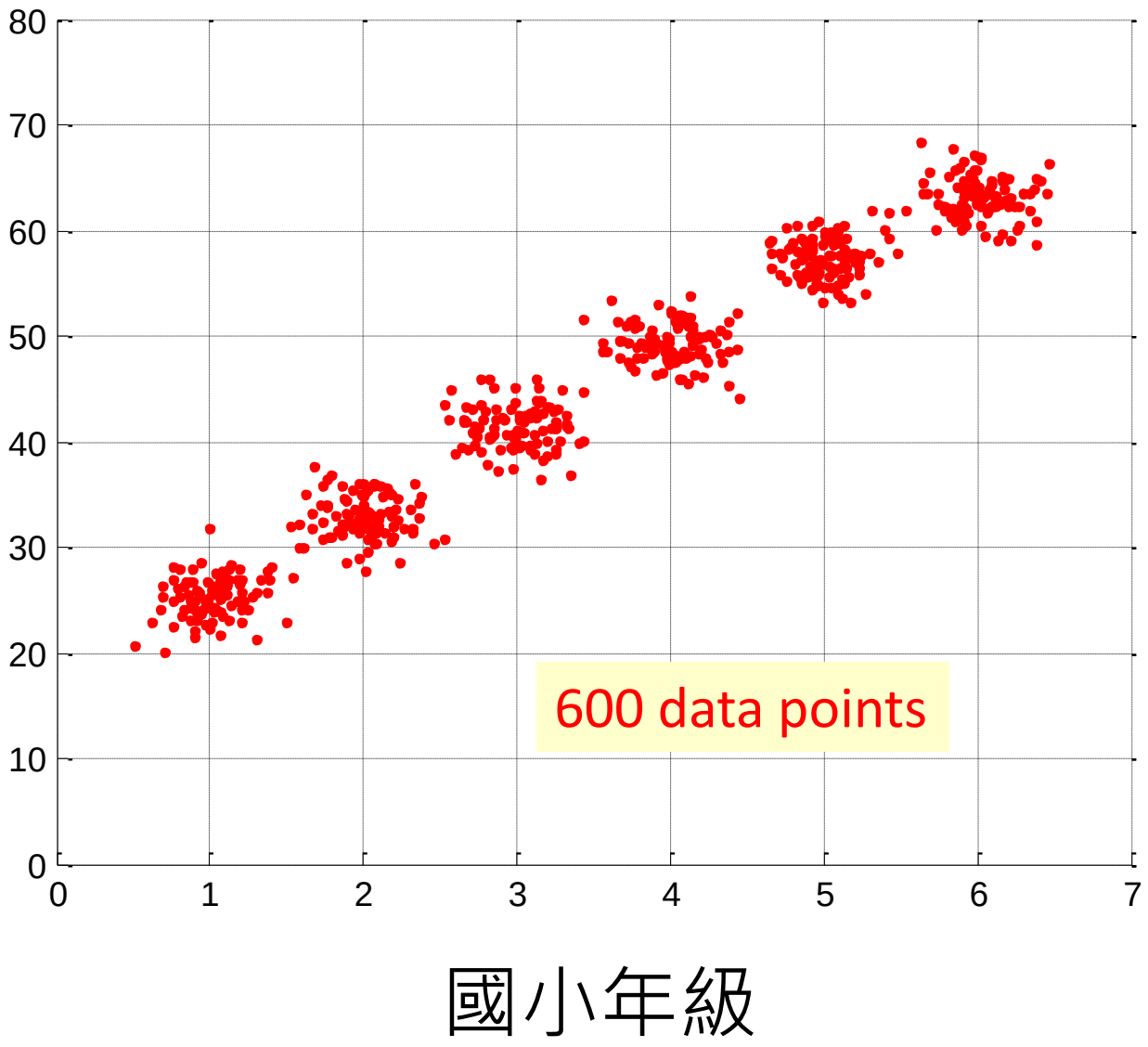
http://statdb.dgbas.gov.tw/pxweb/Dialog/Saveshow.asp

國小學生視力不良率

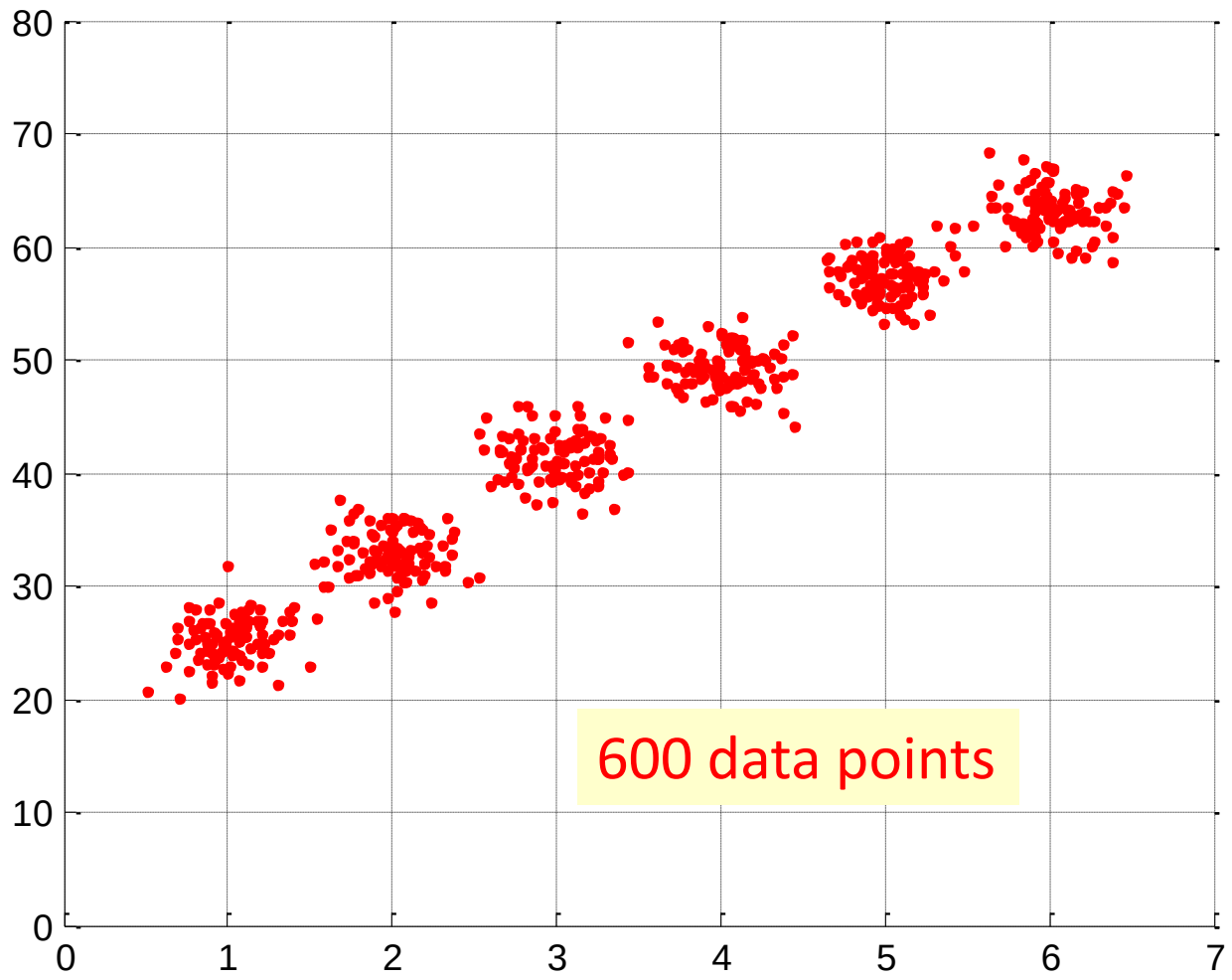


年級	視力不良率(%)
1	25.64
2	32.71
3	41.33
4	49.13
5	57.21
6	63.21

國小學生視力不良率



年級	視力不良率(%)
1	25.64
2	32.71
3	41.33
4	49.13
5	57.21
6	63.21



國小年級

Curve fitting to a linear equation:

$$y = mx + b$$

(Linear regression)

$To\ fit\ a\ linear\ model$: $y = mx + b$

$using\ the\ measured\ data$: $(x_i, y_i)\ for\ i = 1, 2, \ldots N$

$To\ fit\ a\ linear\ model\text{:}\ y = mx + b$

$using\ the\ measured\ data\text{:}\ (x_i, y_i)\ for\ i = 1, 2, \ldots N$

$The\ parameters\ to\ be\ estimated\ are\text{:}\ m\ and\ b$

$Estimated\ parameters\ are\ denoted\ as\text{:}\ \hat{m}\ and\ \hat{b}$

$$For\ every\ x_i, the\ \hat{y}_i\ value\ calculated$$

$$from\ \hat{m}\ and\ \hat{b}\ is: \quad \hat{y}_i = \hat{m}x_i + \hat{b}$$

$For\ every\ x_i, the\ \hat{y}_i\ value\ calculated$

$from\ \hat{m}\ and\ \hat{b}\ is: \quad \hat{y}_i = \hat{m}x_i + \hat{b}$

$The\ error\ between\ the\ measured\ data, y_i,$

$and\ calculated\ value, \hat{y}_i, is\ (y_i - \hat{y}_i)$

The mean squared error is:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

$$\boxed{\hat{y}_i = \hat{m}x_i + \hat{b}}$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{m}x_i - \hat{b}\right)^2$$

$$The\ minimum\ of\ MSE\ occurs\ at$$

$$\frac{\partial}{\partial m} MSE = 0 \ \ and \ \ \ \frac{\partial}{\partial b} MSE = 0$$

$$\frac{\partial}{\partial m} MSE = \frac{1}{N} \sum_{i=1}^{N} \left(-2x_i\right)\left(y_i - \hat{m}x_i - \hat{b}\right)$$

$$\frac{\partial}{\partial b} MSE = \frac{1}{N} \sum_{i=1}^{N} \left(-2\right)\left(y_i - \hat{m}x_i - \hat{b}\right)$$

```
def linear_regression(x, y, m_current=0, b_current=0, epochs=1000,
learning_rate=0.0001):
```

Initial guess

```
    N = float(len(y))
    for i in range(epochs):
        y_current = (m_current * x) + b_current
        cost = sum([data**2 for data in (y - y_current)]) / N
        m_gradient = -(2/N) * sum(x * (y - y_current))
        b_gradient = -(2/N) * sum(y - y_current)
        m_current = m_current - (learning_rate * m_gradient)
        b_current = b_current - (learning_rate * b_gradient)
    return m_current, b_current, cost
```

https://towardsdatascience.com/linear-regression-using-gradient-descent-in-10-lines-of-code-642f995339c0

jupyter Python_Optimize_Near_Sightedness

File Edit View Insert Cell Kernel Widgets Help

Run Code

國小兒童近視比例 - 第2個方法

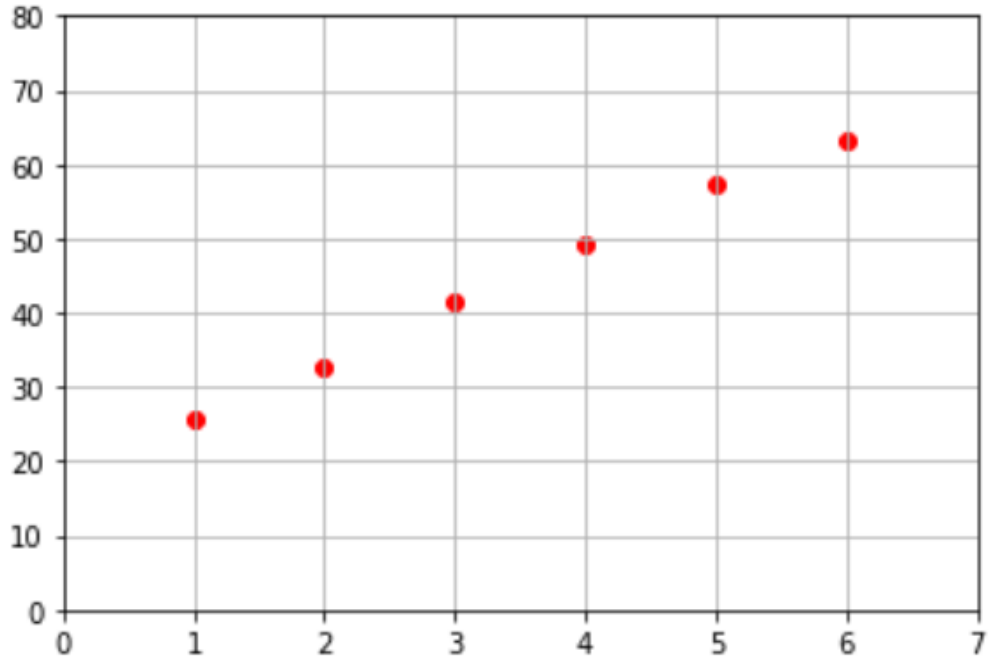
Optimization

In [1]:
```
import math
import numpy as np
import matplotlib.pyplot as plt
```

輸入資料

In [2]:
```
x = np.array([1.0,2.0,3.0,4.0,5.0,6.0])
y = np.array([25.64,32.71,41.33,49.13,57.21,63.21])
print(x.dtype, x.size, y.dtype, y.size)
plt.plot(x,y,'ro')
plt.xlim([0,7])
plt.ylim([0,80])
plt.grid()
```

```
float64 6 float64 6
```



製造雜訊

In [3]:
```
mu, sigma = 0, 1 # mean and standard deviation
x_noise = 0.2*np.random.normal(mu, sigma, 600)
y_noise = 2.0*np.random.normal(mu, sigma, 600)
```

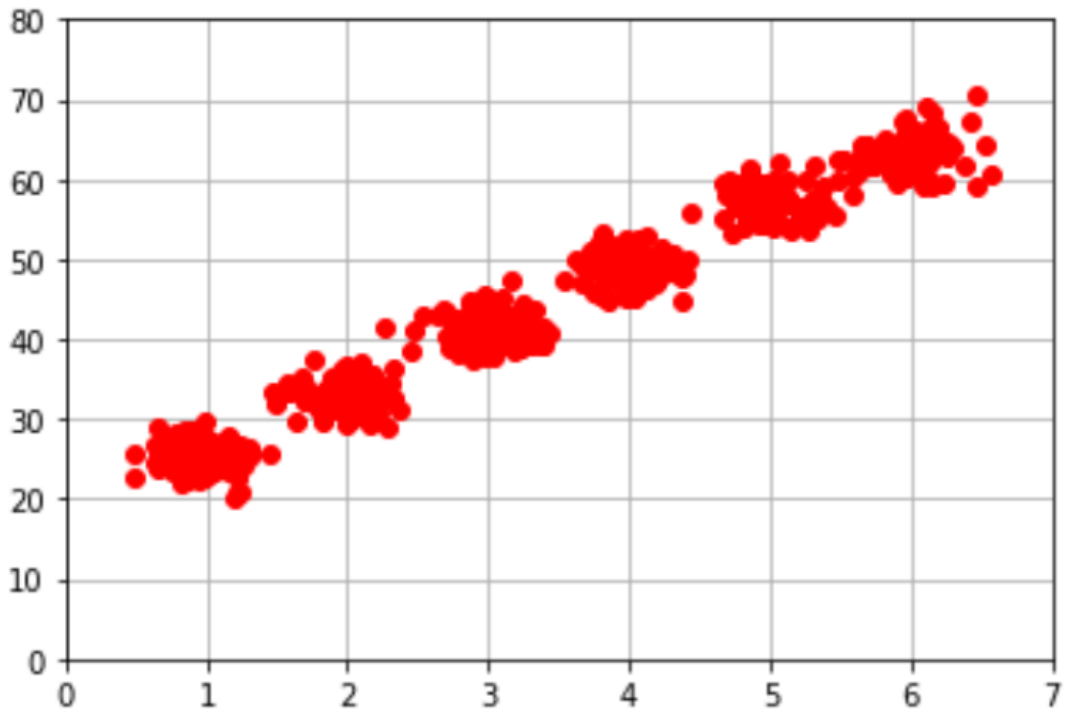
將雜訊加到原始數據

In [4]:
```
x_new = np.zeros(600)
y_new = np.zeros(600)
for i in range(0,600):
    r1 = int(i/100)
    x_new[i] = x[r1] + x_noise[i]
    y_new[i] = y[r1] + y_noise[i]

x = x_new
y = y_new
print(x.dtype, x.size, y.dtype, y.size)

plt.plot(x,y,'ro')
plt.xlim([0,7])
plt.ylim([0,80])
plt.grid()
```

```
float64 600 float64 600
```



設定初始值

In [5]:
```
m_current=0
b_current=0
epochs=9000
learning_rate=0.001
print(len(y))
```

```
600
```

做線性回歸

In [6]:
```
N = float(len(y))

for i in range(epochs):
    y_current = (m_current * x) + b_current
    cost = sum([data**2 for data in (y - y_current)]) / N
    m_gradient = -(2/N) * sum(x * (y - y_current))
    b_gradient = -(2/N) * sum(y - y_current)
    m_current = m_current - (learning_rate * m_gradient)
    b_current = b_current - (learning_rate * b_gradient)
#    print("Iteration",i,"\nm value is",m_current,",  b valu
```

列印結果

In [7]:
```
print(m_current,b_current)
print("\n")
print("Y = ",m_current," * X + ",b_current)
```

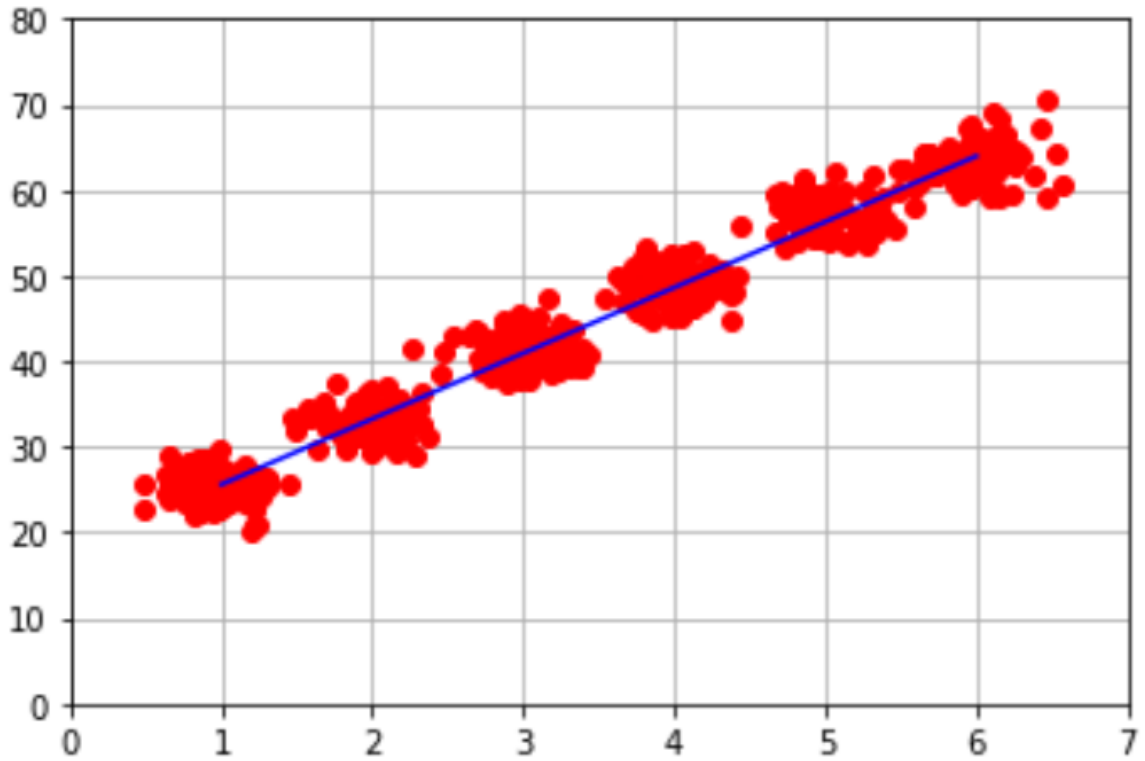
```
7.806342469055911 17.524682463404588


Y =  7.806342469055911  * X +  17.524682463404588
```

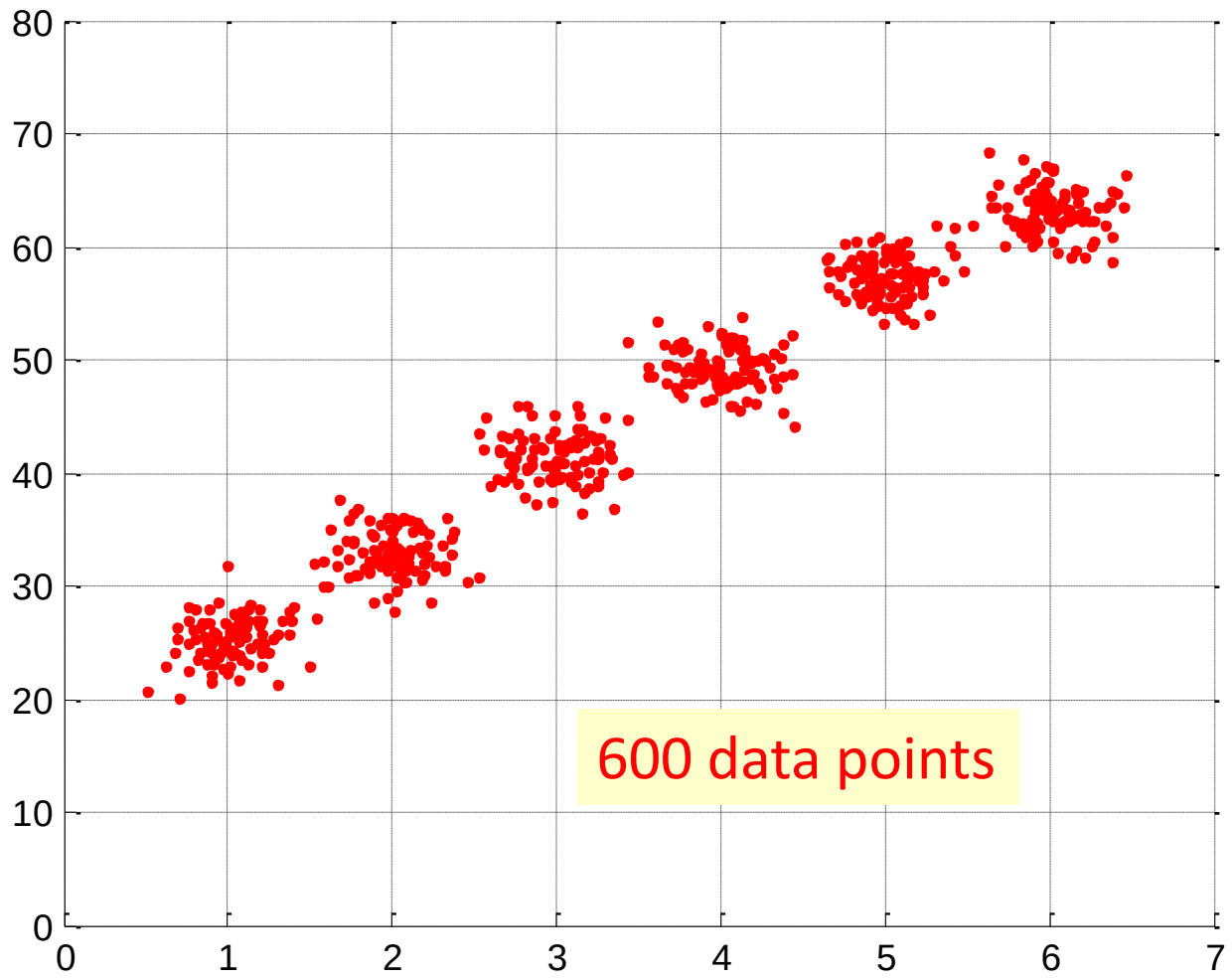
Plot the fitted line

In [8]:
```
x_fit = np.zeros(2)
y_fit = np.zeros(2)
x_fit[0] = 1
x_fit[1] = 6
y_fit[0] = m_current*1 + b_current
y_fit[1] = m_current*6 + b_current
```

In [9]:
```
plt.xlim([0,7])
plt.ylim([0,80])
plt.plot(x,y,'ro')
plt.plot(x_fit,y_fit,'b-')
plt.grid()
plt.show()
```



Review the linear regression problem:

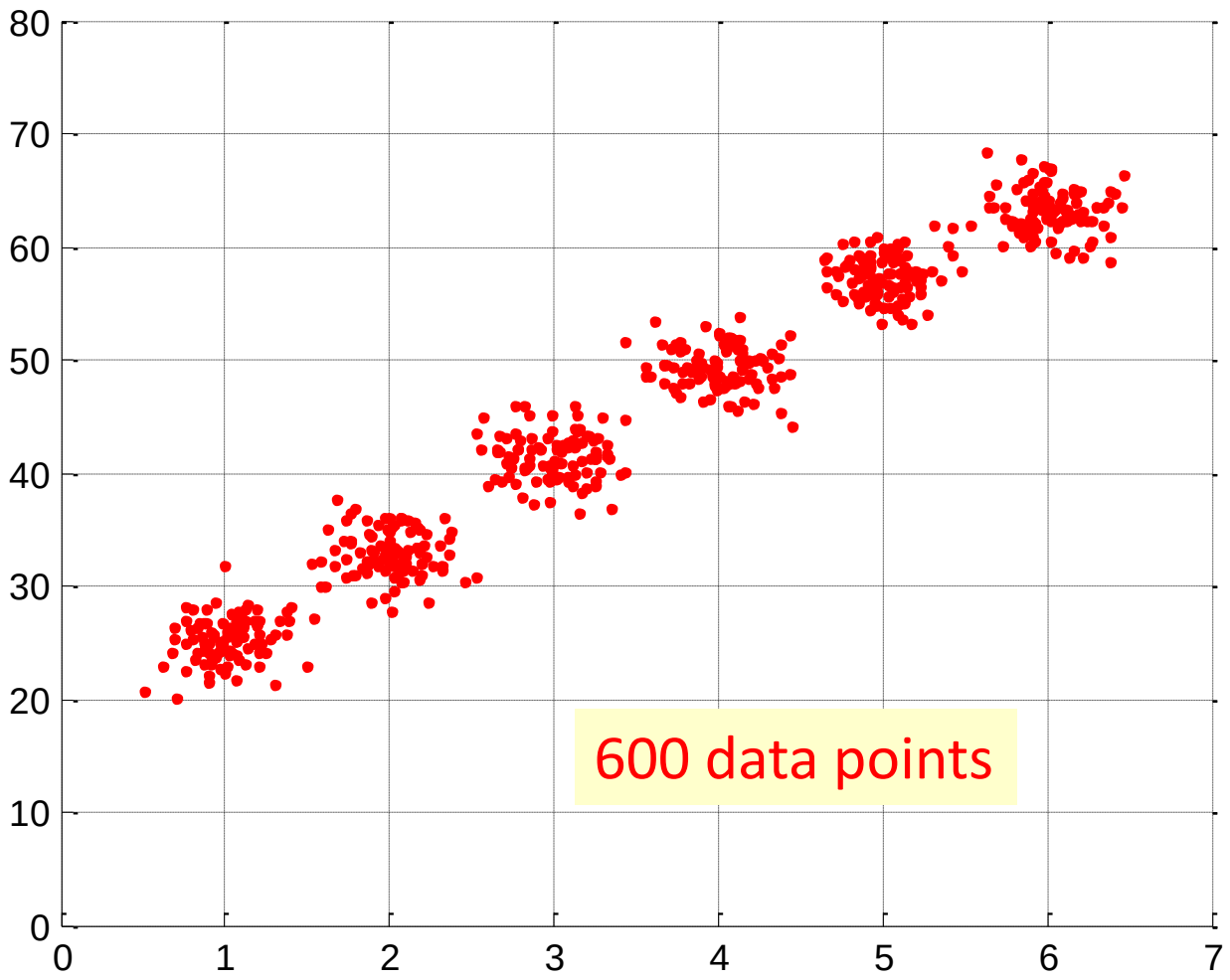


國小年級

Curve fitting to a linear equation:

$$y = mx + b$$

(Linear regression)



國小年級

Curve fitting to a linear equation:

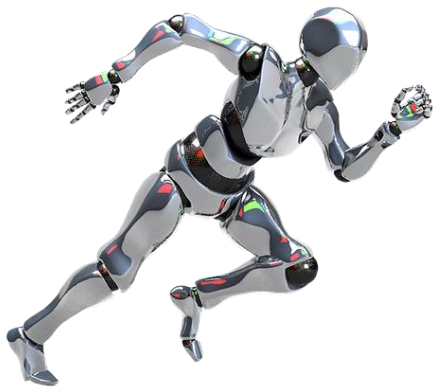
$$y = mx + b$$

$$y = wx + b$$

(Linear regression)

$$f(x) = y = wx + b$$

(Linear regression)



$$\boldsymbol{f(x_i, w_i, b) = \sum_{i=1}^{n} x_i w_i + b}$$

Homework(回家看影片):

ML Lecture 1: Regression - Case Study

https://www.youtube.com/watch?v=fegAeph9UaA&index=3&list=PLJV_el3uVTsPy9oCRY30oBPNLCo89yu49

ML Lecture 1: Regression - Demo

https://www.youtube.com/watch?v=1UqCjFQiiy0&list=PLJV_el3uVTsPy9oCRY30oBPNLCo89yu49&index=4

Thank you for your attention